

PART 0 — Pehle Basics Clear Karte Hain

Chess Engine Actually Kya Hota Hai?

Socho tum ek **grandmaster** ho. Jab tumhari turn aati hai, tum kya karte ho?

1. **Saare possible moves** sochte ho (maybe 30-40 moves)
2. Har move ke baad **opponent ke possible replies** sochte ho (30-40 each)
3. Aur aage bhi... (this creates a **tree** )
4. Har branch ko **score** dete ho ("agar main ye chalaun toh mera position acha hai ya bura")
5. Sabse acha score wala move chunte ho

Bas! Ye hi chess engine hai. Computer bhi yahi karta hai — bas bahut fast, aur bahut deep.

Analogy: Ye aise hi hai jaise tum ek shatranj ka **family tree** bana rahe ho — har node ek position, har branch ek move.

FEN Kya Hai?

FEN = Forsyth–Edwards Notation. Ye ek **single line string** hai jo pura chess board describe karti hai.

text

```
rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq e3 0 1
```

Isko aise padho (5 parts):

Part	Example	Matlab
1	<code>rnbqkbnr/pppppppp/...</code>	8 rows ka board (lowercase = black, UPPERCASE = white)
2	<code>b</code>	Kiski turn hai (<code>b</code> = black, <code>w</code> = white)
3	<code>KQkq</code>	Castling rights (King-side, Queen-side for both)
4	<code>e3</code>	En-passant target square
5	<code>0 1</code>	Halfmove clock & fullmove number

Analogy: FEN = chess position ka **Aadhaar card**. Har position ki unique identity.

Important: Engine ke liye sirf pehle **4 parts** matter karte hain. Last 2 (clocks) ignore karta hai — kyunki same position ho sakti hai clock alag hone pe. Isliye `getPositionKey()` sirf pehle 4 tokens use karta hai.

⚡ Web Worker Kya Hai? (Ye Sabse Important Concept Hai)

Problem: Agar tum heavy calculation **main thread** (jahan UI chalti hai) pe karoge, toh **screen freeze ho jayegi**. Button click nahi hoga, animation ruk jayega — user ko lagega app hang ho gaya.

Solution: Browser mein **2 threads** hote hain:

- **Main Thread:** UI, clicks, rendering
- **Web Worker Thread:** Background heavy math

Dono **messages** bhej ke baat karte hain (`postMessage`).

Analogy: Tum ek **restaurant** chala rahe ho.

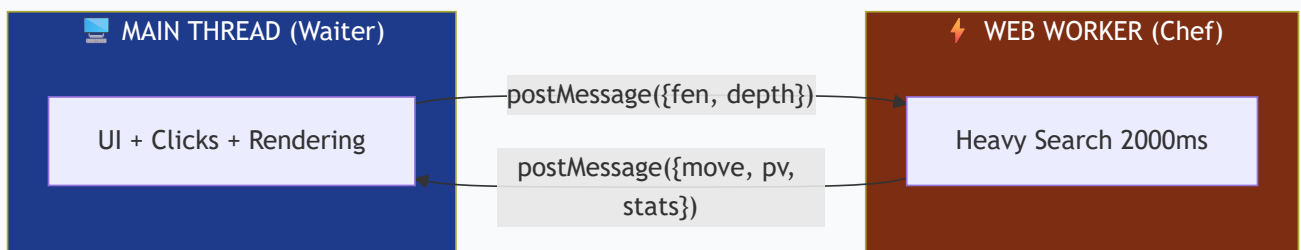
- **Main Thread = Waiter** (customer se baat karta hai, order leta hai, khana serve karta hai — fast & responsive)

- **Web Worker = Chef** (kitchen mein chup-chaap 10 minute khana banata hai, beech mein customer ko dikhta nahi)

Agar waiter khud kitchen mein jaake khana banane lage, toh baaki customers kaun sambhalega? 🤖

Diagram

Code

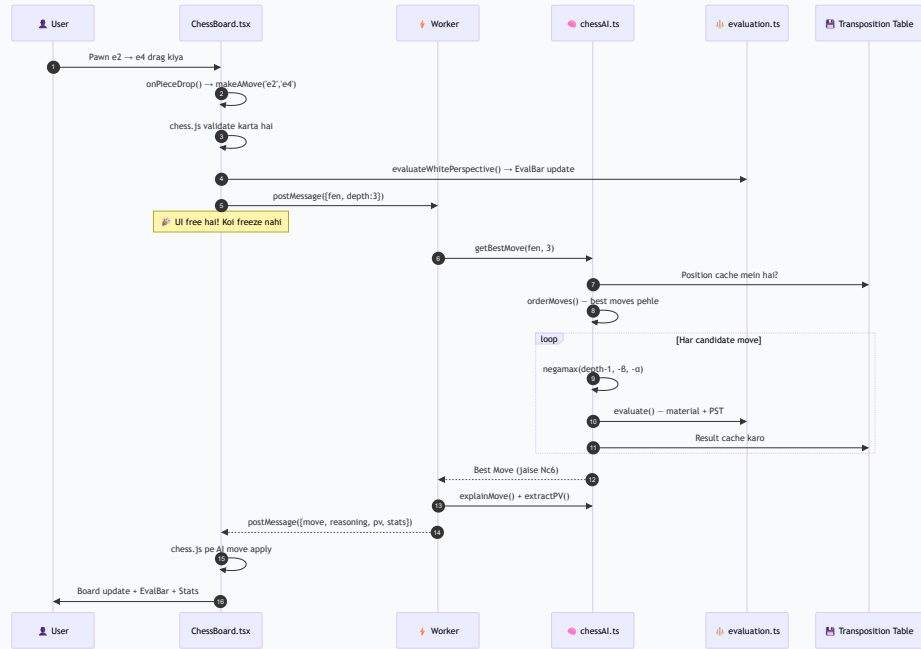


🎬 PART 1 — Ek Move Ka Pura Safar (Step-by-Step)

Maan lo tumne 1. **e4** chala. Ab kya hoga? Chalo **12 steps** mein dekhte hain.

Diagram

Code



Steps Detail Mein:

#	Kya hota hai	Function	File
1	Piece drag-drop detect	<code>onPieceDrop()</code>	ChessBoard.tsx
2	Move validate + execute	<code>makeAMove()</code>	ChessBoard.tsx
3	Score calculate (White ke nazariye se)	<code>evaluateWhitePerspective()</code>	evaluation.ts
4	Worker ko FEN bhejo	<code>postMessage()</code>	ChessBoard.tsx
5	Worker message catch kare	<code>self.onmessage</code>	chessAl.worker.ts
6	Search start	<code>getBestMove()</code>	chessAl.ts
7	Cache check	<code>transpositionTable.get()</code>	transposition.ts
8	Moves sort karo	<code>orderMoves()</code>	chessAl.ts

#	Kya hota hai	Function	File
9	Recursive search	<code>negamax()</code>	chessAI.ts
10	Leaf pe capture search	<code>quiesce()</code>	chessAI.ts
11	Reasoning + PV extract	<code>explainMove()</code> , <code>extractPV()</code>	chessAI.ts
12	UI ko wapas bhejo	<code>postMessage()</code>	chessAI.worker.ts

🧠 PART 2 — Algorithms Ek-Ek Karke (Deep Dive)

Ab main har algorithm ko analogy + diagram + code ke saath samjhata hoon.

🔹 Algorithm 1: Negamax (The Heart of Everything)

🔧 Kya Karta Hai?

Ye recursive function hai jo batata hai: *"Is position mein, jiski turn hai, uske liye best score kya hai?"*

🤔 Ye Negamax Kyu Kehlata Hai? (Minimax se alag kaise?)

Minimax mein 2 functions hote hain: `maximize()` (mera turn) aur `minimize()` (opponent ka turn).

Negamax kehta hai: *"Bhai 2 function kyu? Ek hi kaam chalega!"*

Trick: Zero-sum game mein — **mera profit = opponent ka loss.**

$$\max(A, B) = -\min(-A, -B)$$

Matlab:

- White ka score = `+300`

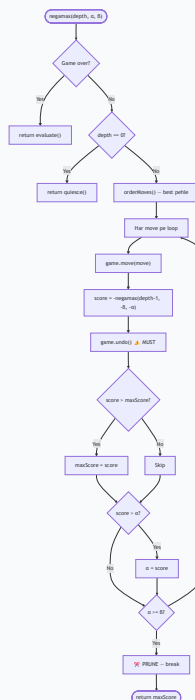
- Black ka score = **-300**
- Ek hi function dono colors ke liye kaam karega, bas **sign flip** karke.

Analogy: Do dost hain, ek hi pizza. Tum 3 slice loge toh dost ko 5 milega, ya tum 5 loge toh dost ko 3. **Tumhara gain = dost ka loss.** Isliye ek hi score likhna kaafi hai — jo tumhare perspective se hai.

Flowchart:

Diagram

Code



Code:

typescript

```

export function negamax(game, depth, alpha, beta, onNode) {
  if (game.isGameOver()) return evaluate(game);
  if (depth === 0) return quiesce(game, alpha, beta, onNode);

  let maxScore = -Infinity;
  const moves = orderMoves(game.moves({ verbose: true }));

  for (const move of moves) {
    game.move(move);
    // ⚠ Notice the MINUS sign - yahi Negamax ki jaan hai!
    const score = -negamax(game, depth + 1, -beta, -alpha, onNode);
  
```

```

game.undo();

if (score > maxScore) maxScore = score;
if (score > alpha) alpha = score;
if (alpha >= beta) break; // Alpha-Beta pruning
}
return maxScore;
}

```

🎬 Mini Example (Depth 2, White's turn):

text

Position: White can play e4 or Nf3

Mera turn (White)

├─ e4 → Opponent ke paas: Nc6 ya e5

| ├─ Nc6 → score = -50 (White ke liye)

| └─ e5 → score = +80 (White ke liye)

| → Opponent best (-50 se best matlab White ke liye worst) = -50

| → e4 ki value for White = -(-50) = +50 ✅

|

└─ Nf3 → Opponent ke paas: Nc6 ya d5

 ├─ Nc6 → score = +30

 └─ d5 → score = +20

 → Opponent best = +20 (White ke liye best for opponent)

 → Nf3 ki value = -(+20) = -20 ❌

Best move = e4 (score +50) 🏆

💠 Algorithm 2: Alpha-Beta Pruning (Search Fast Karne Ka Jadoo)

🎯 Kya Karta Hai?

Search tree ki **un branches ko skip** kar deta hai jahan already prove ho chuka hai ki "yahan se kuch acha nahi milega". **~70% time bach jaata hai**, bina answer badle.

🗨️ Two Magic Numbers:

- **α (alpha):** "Main (maximizer) already itna acha score jeet chuka hoon, isse kam nahi lunga"

- β (beta): "Opponent (minimizer) already itna kam score de raha hai, isse zyada nahi dega"

Rule: Agar $\alpha \geq \beta$, toh **cutoff** — matlab ye branch bekaar hai, aage dekhne ka faayda nahi.

Analogy: Tum dukaan pe ho, samaan dekh rahe ho.

- $\alpha = 500$ rupees (Itni best deal mil gayi, isse kam nahi lunga)
- $\beta = 700$ rupees (Shopkeeper itna hi max de sakta hai)
- Ab agar koi item **800 rupees** dikhaye toh? Matlab dekho — tumhara α (500) already shopkeeper ki limit (700) se compare mein... wait. Chalo simpler:

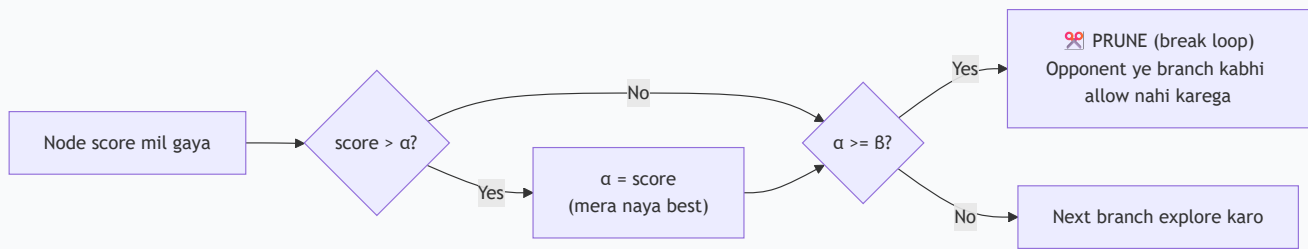
Asli analogy: Tum soch rahe ho "main A hostel jaunga ya B hostel".

- A hostel mein dekha: kamra acha hai, rent 5000 hai ($\alpha = 5000$ established)
- B hostel ka pehla kamra dekha: rent 8000, kuch khaas nahi
- B hostel ka doosra kamra dekhne jaane se pehle: **already pata hai pehla 8000 ka tha**, agar doosra 5000 se acha hoga toh bhi B ka average 5000 se upar hi rahega
- Toh B ke baaki kamre dekhne ka kya faayda? → **PRUNE B!** ✓

 **Flowchart:**

Diagram

Code



Code:

typescript

```

if (score > alpha) {
  alpha = score; // Naya best mila
}
if (alpha >= beta) {
  break; // ✂ CUTOFF! Baaki moves check karne ka koi matlab nahi
}
  
```

◆ Algorithm 3: Quiescence Search (Horizon Effect Killer)

Problem Kya Hai?

Maan lo depth 3 pe pahunch gaye. Last move mein **opponent tumhara Queen capture** kar leta hai — but depth 0 ho gayi. Engine sochta hai "arre, main toh aage hi nahi dekh raha, ab score count karte hain".

Ye ghazab ki galti hai! Jaise fight ke beech mein score count karna — **aakhri punch ke baad hi count karo!**

Solution:

Depth 0 pe pahunchne ke baad bhi, **agar captures available hain**, toh unhe search karo. Jab tak board "quiet" (koi capture nahi) na ho jaye.

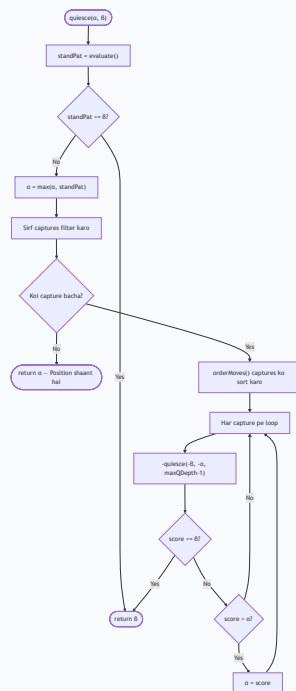
Analogy: Cricket match mein tum 1st innings ke score pe khush ho rahe the...

- **Bina Quiescence:** "50 runs ho gaye, main jeet gaya" 🙌
- **With Quiescence:** "Bhai, 2nd innings abhi baaki hai, chalo poora match khatam karte hain phir bolenge" 🏏

Flowchart:

Diagram

Code



Code:

typescript

```
export function quiesce(game, alpha, beta, onNode, maxQDepth = 6) {
  if (game.isGameOver()) return evaluate(game);

  // Stand-pat: "Agar main kuch nahi karun toh kya score hai?"
  const standPat = evaluate(game);
  if (standPat >= beta) return beta; // Opponent allow nahi karega
  if (standPat > alpha) alpha = standPat;
  if (maxQDepth <= 0) return alpha; // Safety limit
```

```
// Sirf captures search karo (quiet moves skip)
const candidateMoves = game.inCheck()
  ? game.moves({ verbose: true })
  : game.moves({ verbose: true }).filter((m) => !!m.captured);

for (const move of orderMoves(candidateMoves)) {
  game.move(move);
  const score = -quiesce(game, -beta, -alpha, onNode, maxQDepth - 1);
  game.undo();

  if (score >= beta) return beta;
  if (score > alpha) alpha = score;
}
return alpha;
}
```

Note: Check mein ho toh **saare moves** dekhe jaate hain (kyunki check se nikalna zaroori hai, capture ho ya na ho).

◆ Algorithm 4: Move Ordering (MVV-LVA)

🎯 Kya Karta Hai?

Moves ko **smartly sort** karta hai — best moves sabse pehle.

Kyu zaroori hai? Kyunki Alpha-Beta tab sabse fast chalta hai jab **best move pehle** try karo. Cutoff pehle hi ho jaata hai, baaki branches skip ho jaati hain.

MVV-LVA = Most Valuable Victim — Least Valuable Attacker

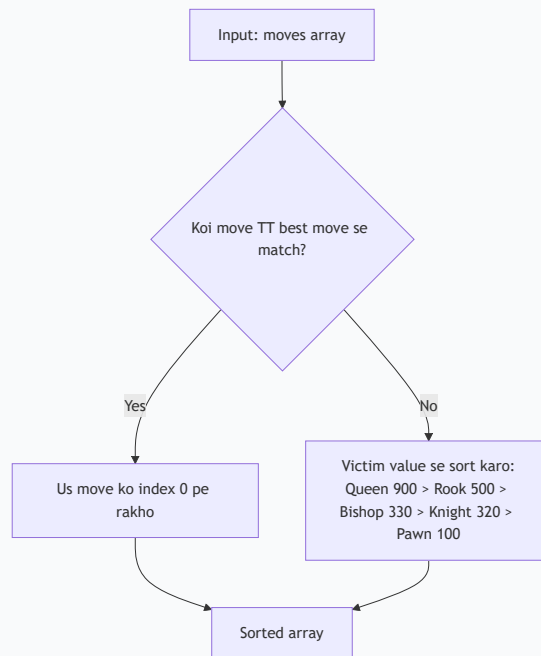
Analogy: Tumhare paas 5 punch hain, opponent ke saamne khada hai.

- **Smart order:** Pehle **jaw pe full power punch** maaro (ko ho sakta hai opponent gir jaye)
- **Dumb order:** Pehle **halka sa jhatka** maaro, phir dekho kya hota hai

📊 Flowchart:

Diagram

Code



Code:

typescript

```

export function orderMoves(moves, ttMove) {
  return [...moves].sort((a, b) => {
    // 1. TT move = highest priority
    if (ttMove) {
      const aIsTT = a.lan === ttMove.lan;
      const bIsTT = b.lan === ttMove.lan;
      if (aIsTT && !bIsTT) return -1;
      if (!aIsTT && bIsTT) return 1;
    }
    // 2. MVV: bade victim wale captures pehle
    const aVal = a.captured ? PIECE_VALUES[a.captured] : 0;
    const bVal = b.captured ? PIECE_VALUES[b.captured] : 0;
    return bVal - aVal;
  });
}

```

Example Sort:

text

Input: [e4, Nxe5 (knight captures pawn), Qxf7 (queen captures pawn), Nf3]
 Output: [Qxf7, Nxe5, e4, Nf3]

↑ Queen capture (pawn = 100, but big attacker) – wait

Hmm, dekho — MVV-LVA actually **victim ki value** dekhta hai, attacker ki nahi. Toh:

text

Qxf7 → captured = pawn (100) → score 100

Nxe5 → captured = pawn (100) → score 100

Dono same score hain, toh original order maintain hoga.

Real MVV-LVA mein attacker ki value bhi count hoti hai (least valuable attacker best hota hai — kyunki agar wapas capture ho gaya toh kam loss). Ye code simplified version hai, sirf victim value dekhta hai.

◆ Algorithm 5: Transposition Table (Caching = Speed)

🎯 Problem:

Chess mein **transposition** bahut common hai. Same position tak **alag-alag move orders** se pahunch sakte ho:

text

Path 1: 1. d4 Nf6 2. c4

Path 2: 1. c4 Nf6 2. d4

↑ Same position!

Bina cache ke, engine **same position ko 2 baar search** karega — waste!

🎯 Solution:

Har position ki **FEN hash** → **score** memory mein store karo. Agar same position dobara aaye, **instant return**.

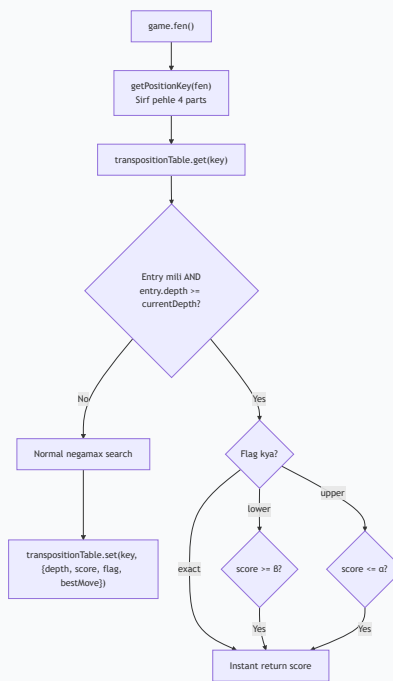
Analogy: Tumhare paas maths ke difficult sawaal ka solution **rough copy** mein likha hai. Exam mein same sawaal aaya toh **pura solve karne ki zaroorat nahi** — copy se answer likh do. Time bachao! 🕒

🗨 TT Entry Ke 3 Flags (Ye Important Hai):

Flag	Matlab	Kab Use Hota Hai
exact	Pura accurate score	Normal case
lower	Score kam se kam itna toh hai (β cutoff hua tha)	Search mein prune hua
upper	Score zyada se zyada itna hai (α fail-low)	Search incomplete

Diagram

Code



Code:

typescript

```
// CHECK (negamax ke starting mein):
const key = getPositionKey(game.fen());
const entry = transpositionTable.get(key);

if (entry && entry.depth >= depth) {
  if (entry.flag === 'exact') return entry.score;
  if (entry.flag === 'lower' && entry.score >= beta) return entry.score;
  if (entry.flag === 'upper' && entry.score <= alpha) return entry.score;
```

```

}

// STORE (Loop ke baad):
let flag: TTFlag = 'exact';
if (maxScore <= origAlpha) flag = 'upper';
else if (maxScore >= beta) flag = 'lower';

transpositionTable.set(key, { depth, score: maxScore, flag, bestMove });

```

getPositionKey() Ka Trick:

```

typescript

export function getPositionKey(fen: string): string {
  return fen.split(' ').slice(0, 4).join(' ');
  // ↑ Clocks (parts 4,5) hata diye!
}

```

Kyu? Kyunki same position **alag move numbers** pe bhi identical hoti hai. Agar clocks include karega toh cache **kabhi hit nahi hoga**.

◆ Algorithm 6: Evaluation (Engine Ki Aankhein)

Kya Karta Hai?

Position ka **score** deta hai (centipawns mein — 100 = 1 pawn).

2 Components:

1. **Material:** Pieces ki values
2. **PST (Piece-Square Tables):** Piece kahan khada hai, uska bonus/penalty

Piece Values (Standard):

Piece	Value (centipawns)
Pawn (p)	100
Knight (n)	320

Piece	Value (centipawns)
Bishop (b)	330
Rook (r)	500
Queen (q)	900
King (k)	∞ (game over)

🎯 PST Kya Hai?

Har piece ke liye **8x8 grid** hoti hai jisme har square ka bonus likha hai.

Knight PST (example):

text

```
-50 -40 -30 -30 -30 -30 -40 -50
-40 -20  0  0  0  0 -20 -40
-30  0 10 15 15 10  0 -30
-30  5 15 20 20 15  5 -30
-30  0 15 20 20 15  0 -30
-30  5 10 15 15 10  5 -30
-40 -20  0  5  5  0 -20 -40
-50 -40 -30 -30 -30 -30 -40 -50
```

Dekho — **center mein +20 bonus, corner mein -50 penalty.**

Analogy: Soldier gali ke corner mein khada hai toh bekaar hai — kuch nahi kar sakta.

Chowk (center) pe khada hai toh pura area control karega! 🎯

🎯 Endgame King Table Alag Hoti Hai:

Opening/Middle game: King **corner mein safe** rehta hai (PST mein corner bonus)

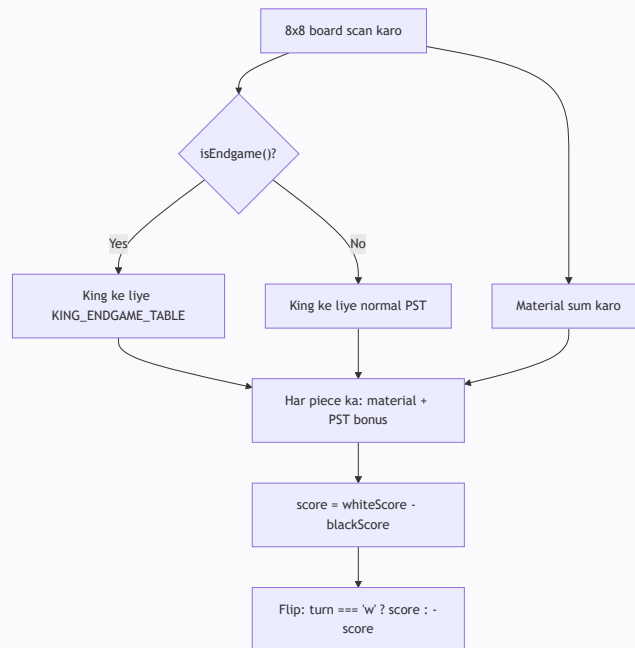
Endgame: King **center mein active** hona chahiye (attack + support ke liye)

Isliye `isEndgame()` check hota hai — queens trade ho gayi? Toh `KING_ENDGAME_TABLE` use karo.

📊 Flowchart:

Diagram

Code



Code:

typescript

```

export function evaluate(game: Chess): number {
  if (game.isCheckmate()) return -Infinity;
  if (game.isDraw()) return 0;

  let whiteScore = 0, blackScore = 0;
  const board = game.board();
  const endgame = isEndgame(game);

  for (let r = 0; r < 8; r++) {
    for (let c = 0; c < 8; c++) {
      const piece = board[r][c];
      if (!piece) continue;

      const materialVal = PIECE_VALUES[piece.type] ?? 0;
      const table = (piece.type === 'k' && endgame)
        ? KING_ENDGAME_TABLE
        : PST[piece.type];

      // ⚠ Black ke liye row mirror karo (7 - r)
      const pstVal = piece.color === 'w' ? table[r][c] : table[7 - r][c];
      const totalVal = materialVal + pstVal;
    }
  }
}

```

```

    if (piece.color === 'w') whiteScore += totalVal;
    else blackScore += totalVal;
  }
}

const score = whiteScore - blackScore;
// 🎯 CRITICAL: Side-to-move perspective!
return game.turn() === 'w' ? score : -score;
}

```

⚠ Warning (Ye Galti Sab Karte Hain):

Agar tum **last line** bhool gaye, toh engine **suicide moves** chalega! Kyunki Negamax expect karta hai ki *"score hamesha current player ke liye positive ho"*.

◆ Algorithm 7: Iterative Deepening (Time Management)

🎯 Problem:

Direct Depth 5 search karoge toh kabhi 2 sec, kabhi 30 sec lag sakta hai. **Time predict nahi hota.**

🎯 Solution:

Pehle **Depth 1**, phir **Depth 2**, phir **Depth 3**... Har step pe timer check karo. Jab time khatam, **pichli complete depth ka best move** return kar do.

Bonus: Pichle depth ka best move agli depth pe **index 0** pe rakho — isse search super fast hoti hai!

Analogy: Exam mein tum:

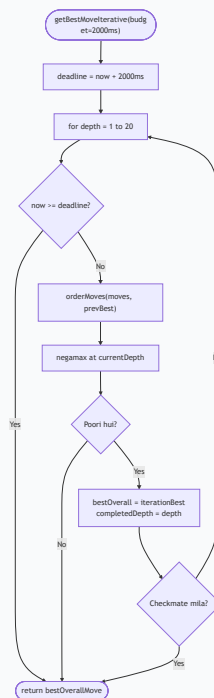
1. Pehle saare **1-mark wale** pakke karo ✅
2. Phir **2-mark wale** ✅
3. Phir **5-mark wale** (agar time bache) ⌚

Agar time khatam, tumhare paas **kam se kam 1-mark wale** toh pakke hain — zero nahi!

Flowchart:

Diagram

Code



Code:

typescript

```
export function getBestMoveIterative(fen, timeLimitMs = 2000, maxDepth = 20) {
  const game = new Chess(fen);
  const deadline = Date.now() + timeLimitMs;

  let bestOverallMove = legalMoves[0];
  let isTimeUp = false;
  let totalNodes = 0;

  // Lightweight check: har 1024 nodes pe timer dekho
  const checkTime = (): boolean => {
    totalNodes++;
    if ((totalNodes & 1023) === 0 && Date.now() >= deadline) isTimeUp = true;
    return isTimeUp;
  };

  for (let depth = 1; depth <= maxDepth; depth++) {
    if (Date.now() >= deadline) break;

    // 🚩 Previous best move index 0 pe!
```

```

const moves = orderMoves(legalMoves, bestOverallMove);
let iterationCompleted = true;
let iterationBestMove = moves[0];

for (const move of moves) {
  if (checkTime()) { iterationCompleted = false; break; }

  game.move(move);
  const score = -negamax(game, depth - 1, -Infinity, Infinity, checkTime);
  game.undo();

  if (isTimeUp) { iterationCompleted = false; break; }
  if (score > bestScore) { bestScore = score; iterationBestMove = move; }
}

if (iterationCompleted) {
  bestOverallMove = iterationBestMove;
  completedDepth = depth;
} else {
  break; // Adhuri depth discard karo
}

return { bestMove: bestOverallMove, depthReached: completedDepth };
}

```

`(totalNodes & 1023) === 0` — Ye bitwise trick hai: har 1024 nodes pe true hota hai.

`Date.now()` slow hota hai, isliye bar-bar call nahi karte.

PART 3 — Core Data Structures

1 `chessRef.current` — Game Engine Ref

typescript

```
const chessRef = useRef(new Chess());
```

Kya hai: `chess.js` ka live instance — rule validation, move generation, check detection.

Kyu useRef: React re-render nahi karna chahte. Performance.

Analogy: Ye tumhari **physical chess board** hai. Piece hilao — board update hoti hai. Har baar naya board nahi banate.

2 fen State — UI Rendering

```
typescript

const [fen, setFen] = useState("rnbqkbnr/...");
```

Kya hai: FEN string **React state** mein — taaki `react-chessboard` board redraw kar sake.

3 Transposition Table Map

```
typescript

export interface TTEEntry {
  depth: number;
  score: number;
  flag: 'exact' | 'lower' | 'upper';
  bestMove?: Move | null;
}
```

Kya hai: `Map<string, TTEEntry>` — FEN → cached score.

4 Principal Variation (PV)

```
typescript

pv: string[] = ["Nc6", "Nc3", "Nf6"]
```

Kya hai: Anticipated moves ki line. UI mein "AI aage ye soch raha hai" dikhane ke liye.

5 Worker Messages

```
typescript

// Main → Worker
{ fen: string, depth?: number, timeLimitMs?: number }
```

```
// Worker → Main
{ move, bestMove, depthReached, score, nodes, timeMs, reasoning, pv }
```

⚠️ PART 4 — Beginner Gotchas (Ye Galtiyan Mat Karna!)

❌ Galti #1: `evaluate()` Ka Perspective Flip Bhoolna

typescript

```
// ❌ GALAT — Black ke liye suicide moves
return whiteScore - blackScore;

// ✅ SAHI
const score = whiteScore - blackScore;
return game.turn() === 'w' ? score : -score;
```

Kyu? Negamax har node pe sochta hai *"main maximize kar raha hoon"*. Agar Black ke turn pe White's perspective return karoge, toh Black apne khilaaf khelega!

❌ Galti #2: `game.undo()` Miss Karna

typescript

```
// ❌ GALAT — Board corrupt
game.move(move);
const score = -negamax(depth - 1);
// undo miss! Board pe trial move permanently reh gaya

// ✅ SAHI
game.move(move);
const score = -negamax(depth - 1, -beta, -alpha);
game.undo(); // ← Ye must hai
```

Kyu? Search mein hazaaron trial moves chalte hain. Har branch ke baad board restore karna zaroori hai, warna agli branch galat position pe search karegi.

Analogy: Tum kisi room mein furniture hilake dekh rahe ho — "sofa yahan acha lagega kya?". Dekhne ke baad **wapas original jagah** rakhna zaroori hai. Warna ghar ka sab kuch bikhra rahega. 🏠

❌ Galti #3: Loop Mein `new Chess(fen)` Banana

typescript

```
// ❌ GALAT - 10x-20x slow!
for (const m of moves) {
  const nextGame = new Chess(game.fen());
  nextGame.move(m);
}

// ✅ SAHI - Ek hi instance pe mutate + undo
for (const m of moves) {
  game.move(m);
  // ... search ...
  game.undo();
}
```

Kyu? Har iteration mein `new Chess()` = 1000s of objects allocate. Search millions of nodes karta hai → **crash level slow**.

❌ Galti #4: FEN Clocks TT Key Mein Daalna

typescript

```
// ❌ GALAT - Cache kabhi hit nahi hoga
const key = fen;

// ✅ SAHI - Sirf pehle 4 parts
const key = fen.split(' ').slice(0, 4).join(' ');
```

❌ Galti #5: Main Thread Pe Direct Search Chalana

typescript

```
// ❌ GALAT — UI freeze!  
const bestMove = getBestMove(fen, 4); // 2000ms freeze 🤖  
  
// ✅ SAHI — Worker mein bhejo  
workerRef.current?.postMessage({ fen, depth: 4 });
```

🚀 PART 5 — Naya Feature Kahan Add Karein?

Feature Idea	File	Function
Bishop pair bonus, Doubled pawns penalty	<code>evaluation.ts</code>	<code>evaluate()</code>
Killer moves, History heuristic	<code>chessAI.ts</code>	<code>orderMoves()</code> , <code>negamax()</code>
Naya telemetry stat (TT hits count)	worker + UI	<code>chessAI.worker.ts</code> + <code>AIStats.tsx</code>
Naya difficulty level	<code>ChessBoard.tsx</code>	difficulty selector
Better <code>explainMove</code> (fork, pin detection)	<code>chessAI.ts</code>	<code>explainMove()</code>
Sound effects	<code>ChessBoard.tsx</code>	<code>makeAMove()</code>

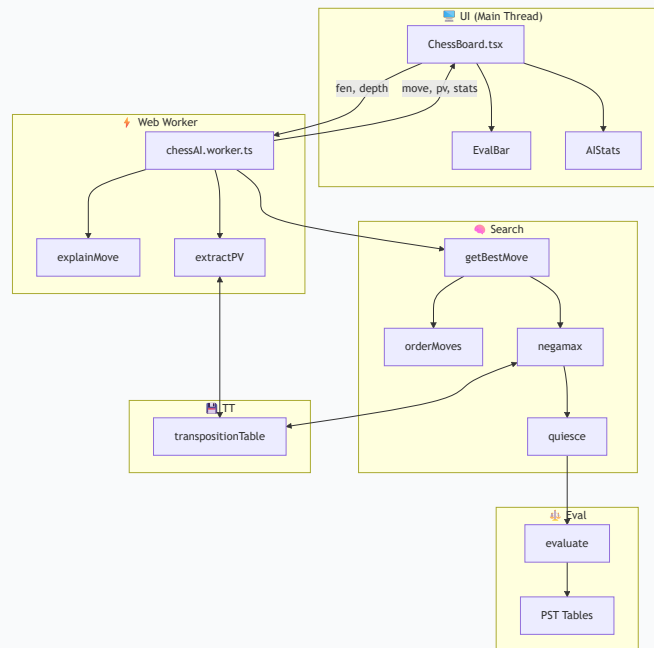
🎯 TL;DR — Ek Line Mein

User ke move pe, Web Worker background mein FEN lekar Negamax search chalata hai. Har node pe Piece-Square Tables se score milta hai, Transposition Table se purani positions reuse hoti hain, Alpha-Beta se useless branches prune hoti hain, aur Quiescence se capture sequences shaant hoti hain — result UI ko wapas jaata hai bina screen freeze kiye.

Poora System Ek Nazar Mein

Diagram

Code



Bhai, agar ab bhi kuch clear nahi hua — batao **kis part pe atke ho**, main usko **aur simple analogy** ke saath samjha dunga! 🚀

This response is AI-generated, for reference only.



use client;.txt

TXT 19.78KB

same for this also

 Thought for 1 second >

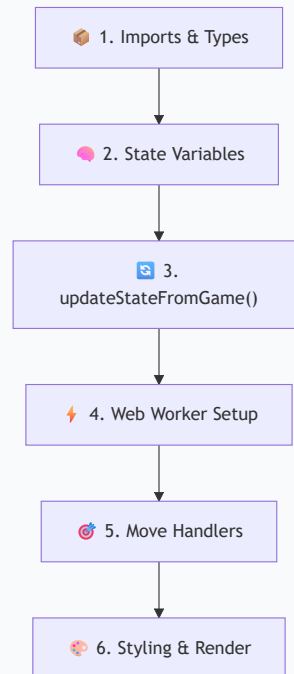
ChessBoard.tsx — Pura Component Line-by-Line (Beginner Hinglish Guide)

Bhai, ye file **project ka dil** hai — jahan user ka click/drag aata hai, jahan se AI trigger hoti hai, aur jahan sab kuch UI pe dikhta hai.

Chalo isko **6 hisson** mein todte hain:

Diagram

Code



PART 1 — Imports & Type Definitions

typescript

```
"use client";

import React, { useState, useRef, useEffect, useCallback } from "react";
import { Chess, type Square } from "chess.js";
import { Chessboard } from "react-chessboard";
import EvalBar from "./EvalBar";
import AIStats from "./AIStats";
import { evaluateWhitePerspective } from "@lib/ai/evaluation";
```

"use client" — Ye Kya Hai?

Next.js **Server Components** by default use karta hai (jahan `useState`, `useEffect` nahi chalta). Jab tumhe **browser pe chalne wala code** chahiye (clicks, state, Web Worker), tab file ke top pe `"use client"` likhna padta hai.

Analogy: Server Component = **restaurant ka kitchen** (chupchaap khana banata hai, customer se interact nahi karta). Client Component = **waiter** (customer se baat karta hai, order leta hai, plate rakhta hai). `"use client"` = *"Ye waiter hai, ise customer se milne do!"* 🧑

Imports Ka Matlab:

Import	Kya deta hai
<code>Chess</code> (chess.js)	Pura chess engine — legal moves, check detection, board state
<code>Square</code> (type)	TypeScript type — <code>"e4"</code> , <code>"a1"</code> jaisi strings
<code>Chessboard</code> (react-chessboard)	Ready-made visual board component
<code>EvalBar</code>	Left side wala vertical evaluation bar
<code>AIStats</code>	AI telemetry card (depth, nodes, PV, reasoning)
<code>evaluateWhitePerspective</code>	Static evaluation function

Type Definitions:

typescript

```
export interface GameState {
  isOver: boolean;
  message: string;
  type: "in_progress" | "checkmate" | "stalemate" | "draw";
  winner?: "white" | "black" | "draw";
}

export type DifficultyLevel = 1 | 2 | 3 | 4 | "expert";
```

`GameState` — Game ki current halat. `type` sirf 4 values mein se ek ho sakta hai (TypeScript **union type** — isse typo catch ho jaate hain).

DifficultyLevel — 1, 2, 3, 4 (fixed depth) ya "expert" (iterative deepening 2000ms).

🗨️ PART 2 — State Variables (Component Ki Memory)

typescript

```
const [mounted, setMounted] = useState(false);
const chessRef = useRef(new Chess());
const workerRef = useRef<Worker | null>(null);

const [fen, setFen] = useState(chessRef.current.fen());
const [turn, setTurn] = useState<"w" | "b">("w");
const [inCheck, setInCheck] = useState(false);
const [history, setHistory] = useState<string[]>([]);
const [lastMove, setLastMove] = useState<{ from: Square; to: Square } | null>(null);
const [selectedSquare, setSelectedSquare] = useState<Square | null>(null);
const [possibleMoves, setPossibleMoves] = useState<Square[]>([]);

const [evaluation, setEvaluation] = useState<number>(0);

const [gameMode, setGameMode] = useState<"vs_ai" | "pvp">("vs_ai");
const [difficulty, setDifficulty] = useState<DifficultyLevel>(3);
const [isAIThinking, setIsAIThinking] = useState(false);
const [aiTelemetry, setAiTelemetry] = useState<{...}>({});
const [gameStatus, setGameStatus] = useState<GameStatus>({...});
```

🎯 useRef vs useState — Sabse Important Concept

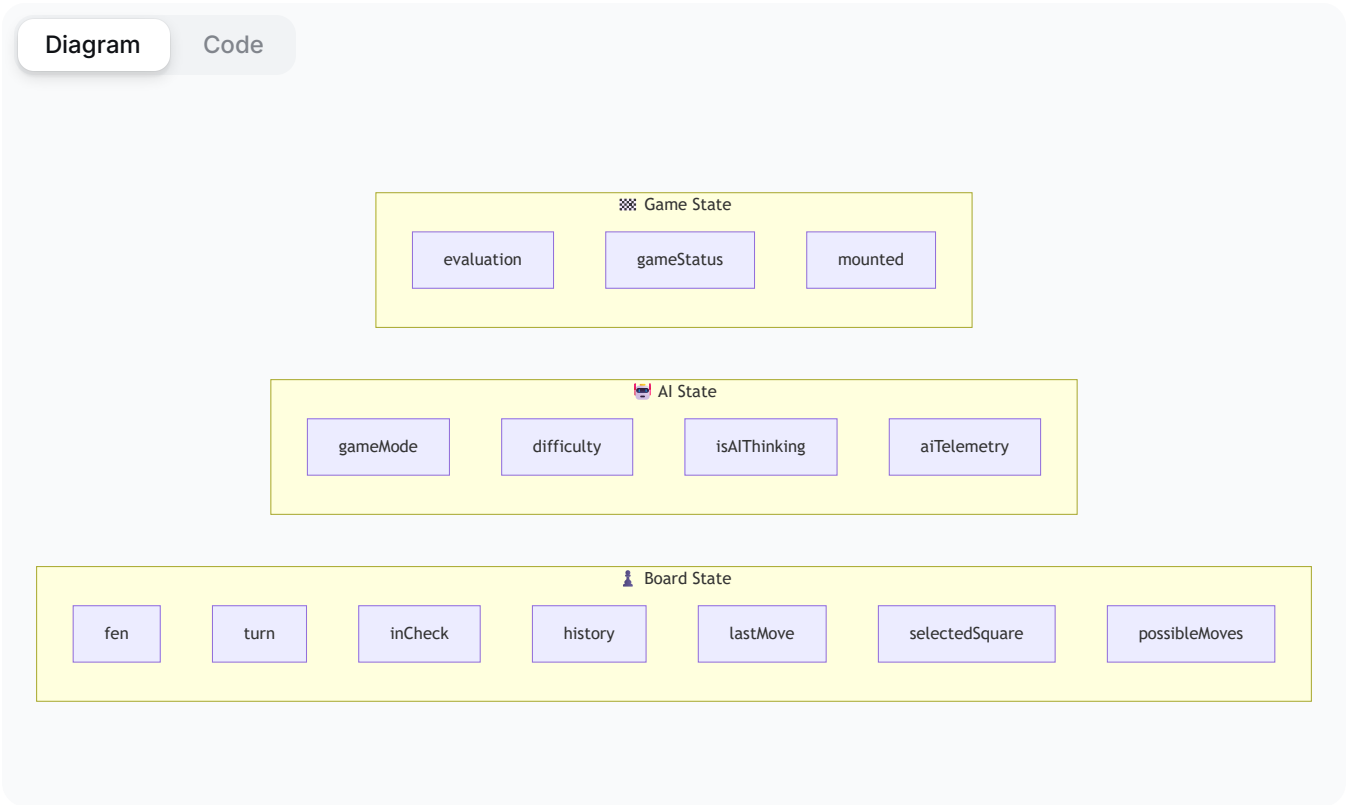
	useState	useRef
Value change pe re-render?	✅ Haan	❌ Nahi
Kis cheez ke liye?	UI data	"Silent" data
Example	fen (board dikhane ke liye)	chessRef (game logic ke liye)

Analogy:

- `useState` = **blackboard** 📝 — jab likhte ho, sab dekh lete hain (re-render)
- `useRef` = **tumhari personal diary** 📁 — tum likhte ho, par baaki log notice nahi karte

Kyu `chessRef` **useRef** mein? Kyunki `chess.js` instance bahut heavy hai. Agar `useState` mein rakhe, toh har move pe **naya instance** banega, aur pura UI re-render hoga. Wasteful!

📁 State Ka Grouping:



Ek-ek State Ka Kaam:

State	Kya Rakhta Hai	Kaun Use Karta Hai
<code>mounted</code>	Component browser pe load hua kya?	Hydration errors rokne ke liye
<code>fen</code>	Board ki FEN string	<code><Chessboard position={fen}></code>
<code>turn</code>	<code>"w"</code> ya <code>"b"</code>	Turn indicator dikhane ke liye
<code>inCheck</code>	Check mein hai kya?	Red glow on king

State	Kya Rakhta Hai	Kaun Use Karta Hai
history	Moves ki list	Move history panel
lastMove	Last move from/to	Yellow highlight
selectedSquare	Click kiya square	Blue highlight
possibleMoves	Legal moves	Green/Red dots
evaluation	White perspective score	EvalBar
gameMode	"vs_ai" ya "pvp"	Controls enable/disable
difficulty	Depth 1-4 ya expert	Worker message mein
isAIThinking	AI soch raha hai?	Board lock + spinner
aiTelemetry	Depth, nodes, PV, reasoning	AIStats card
gameStatus	Checkmate/draw/etc.	Banner

PART 3 — `updateStateFromGame()` (Central Sync Function)

Ye **poora board ka snapshot** leta hai aur React state mein daal deta hai.

typescript

```
const updateStateFromGame = useCallback(() => {
  const game = chessRef.current;
  setFen(game.fen());
  setTurn(game.turn());
  setInCheck(game.inCheck());
  setHistory(game.history());
  setSelectedSquare(null);
  setPossibleMoves([]);
  setEvaluation(evaluateWhitePerspective(game));

  if (game.isCheckmate()) {
    const winner = game.turn() === "w" ? "black" : "white";
```

```

    setStatus({ isOver: true, type: "checkmate", winner, message: `...` });
  } else if (game.isStalemate()) { ... }
  else if (game.isThreefoldRepetition()) { ... }
  else if (game.isInsufficientMaterial()) { ... }
  else if (game.isDraw()) { ... }
  else {
    setStatus({ isOver: false, type: "in_progress", message: `...` });
  }
}, []);

```

🤔 useCallback Kyu?

`useCallback` function ko "**freeze**" kar deta hai — dependencies change na ho toh **same function reference** return karta hai.

Kyu zaroori? Kyunki ye function `useEffect` ke dependency array mein hai. Agar har render pe naya function bane, toh `useEffect` **infinite loop** mein chala jayega!

Analogy: Tumhare paas ek **recipe card** hai. `useCallback` kehta hai: *"Bhai, agar ingredients same hain toh naya card mat print kar — purana hi use kar."* ♻️

`[]` = empty dependency array = **kabhi naya function nahi banega**.

🎯 Checkmate Winner Logic:

typescript

```
const winner = game.turn() === "w" ? "black" : "white";
```

Tricky part! Jab checkmate hota hai, **jiski turn hai wahi HAARA hai**.

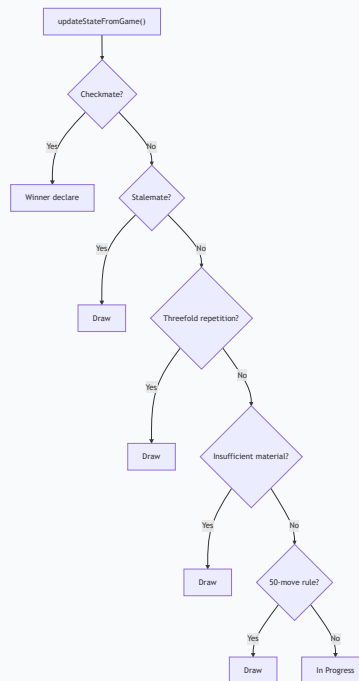
- Agar White ki turn hai aur checkmate → **White haar gaya** → **Black jeeta**
- Isliye `turn === "w" ? "black" : "white"` — ulta logic!

Analogy: Tumhari turn hai, par tum koi legal move nahi kar sakte. Matlab **tum phas gaye** — tum haare. 🚩

🎯 Game Over Ki Priorities (Importance Order):

Diagram

Code



Order kyu matter karta hai? Checkmate pehle check karo — kyunki stalemate aur 50-move rule **checkmate ke baad matter nahi karte**.

⚡ PART 4 — Web Worker Setup (The `useEffect`)

typescript

```

useEffect(() => {
  setMounted(true);

  const worker = new Worker(
    new URL("../workers/chessAI.worker.ts", import.meta.url)
  );
  workerRef.current = worker;

  worker.onmessage = (e: MessageEvent) => {
    const move = e.data?.move || e.data?.bestMove;
    if (move && !chessRef.current.isGameOver()) {
      try {
        const appliedMove = chessRef.current.move(move);
        if (appliedMove) {
          setLastMove({ from: appliedMove.from as Square, to: appliedMove.to as Square

```

```

});

    updateStateFromGame();
    setAiTelemetry({
      depth: e.data.depthReached,
      nodesEvaluated: e.data.nodes,
      searchTimeMs: e.data.timeMs,
      reasoning: e.data.reasoning,
      pv: e.data.pv,
    });
  }
} catch (err) {
  console.error("Failed to apply worker move:", err);
}
}
setIsAIThinking(false);
};

worker.onerror = (err) => {
  console.error("Chess AI Worker error:", err);
  setIsAIThinking(false);
};

return () => {
  worker.terminate();
  workerRef.current = null;
};
}, [updateStateFromGame]);

```

Ye useEffect Kya Karta Hai?

1. **Component mount hone pe** ek Web Worker spawn karta hai
2. Worker se **messages receive** karne ka setup karta hai
3. Component **unmount hone pe** worker ko terminate karta hai (**cleanup**)

NEW

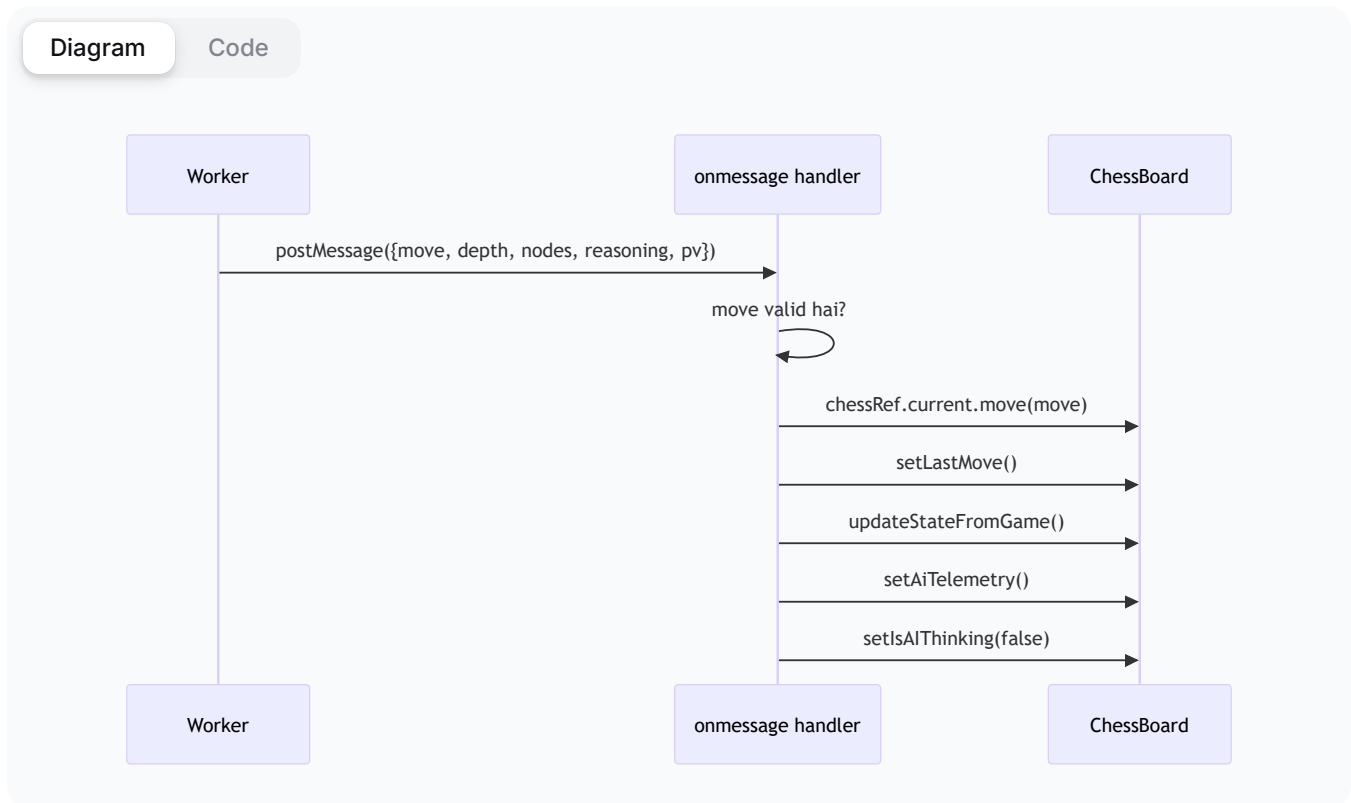
new URL("...", import.meta.url) — Ye Kya Jadoo Hai?

Ye **Vite/Next.js** ka **special syntax** hai Web Workers ke liye. Iske bina bundler worker file ko properly compile nahi kar payega.

Analogy: Tumhare ghar mein ek **alag kitchen** banani hai.

`new URL(path, import.meta.url)` kehta hai *"Ye file mere relative hai — isko alag kitchen (thread) mein chala do!"* 🔍

📁 `worker.onmessage` — Message Handler



⚠️ `try/catch` Kyu?

Agar worker **illegal move** bhej de (jo kabhi nahi hona chahiye, but bugs happen), toh `game.move()` **exception throw** karega. `try/catch` usse UI crash hone se bachata hai.

🧹 Cleanup — `worker.terminate()`

typescript

```
return () => {  
  worker.terminate();  
  workerRef.current = null;  
};
```

Jab user **dusre page pe navigate** kare, ye cleanup chalta hai. Worker **bandh ho jaata hai** — warna background mein **leak** hota rehta.

Analogy: Ghar se nikalte waqt **gas band karna** 🔒 — warna chulha jalta rehta hai.
Same — component unmount = worker ko bandh karo.

🔗 Dependency Array [updateStateFromGame]

Kyu? Kyunki `updateStateFromGame` `useCallback` se wrapped hai (stable reference). Agar ye change ho (jo nahi hoga), toh worker re-setup hoga.

🎯 PART 5 — Move Handlers (User Interaction)

5.1 `makeAMove()` — Core Move Executor

typescript

```
const makeAMove = (from: Square, to: Square): boolean => {
  const game = chessRef.current;

  // 🛑 Guard clauses
  if (game.isGameOver() || isAIThinking) return false;
  if (gameMode === "vs_ai" && game.turn() !== "w") return false;

  try {
    const move = game.move({ from, to, promotion: "q" });
    if (!move) return false;

    setLastMove({ from, to });
    updateStateFromGame();

    // 🤖 AI Trigger
    if (gameMode === "vs_ai" && game.turn() === "b" && !game.isGameOver()) {
      setIsAIThinking(true);
      if (difficulty === "expert") {
        workerRef.current?.postMessage({ fen: game.fen(), isExpert: true, timeLimitMs: 2000 });
      } else {
        workerRef.current?.postMessage({ fen: game.fen(), depth: difficulty });
      }
    }
  }
}
```

```

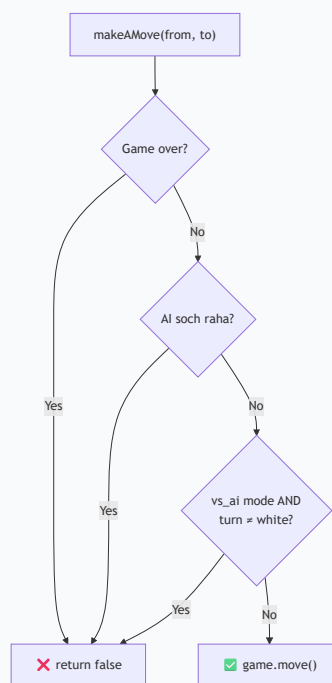
    }
  }
  return true;
} catch {
  return false;
}
};

```

🛡️ Guard Clauses (Bouncer Logic)

Diagram

Code



Analogy: Nightclub ka **bouncer** 🚶 :

- Club band ho gaya? → Andar mat aao
- VIP (AI) busy hai? → Wait karo
- Tum galat time pe aaye? → Baad mein aao
- Sab theek? → **Welcome!**



promotion: "q" — Hamesha Queen Kyu?

Jab pawn last rank pe pahunchti hai, player ko choose karna hota hai: Queen, Rook, Bishop, ya Knight. Ye code **automatically Queen** choose karta hai (99% cases mein best

choice).

Improvement idea: Ek **promotion dialog** dikhao jahan user choose kare.

AI Trigger Logic:

typescript

```
if (gameMode === "vs_ai" && game.turn() === "b" && !game.isGameOver()) {
  setIsAIThinking(true);
  // Expert vs Fixed Depth
  if (difficulty === "expert") {
    workerRef.current?.postMessage({ fen: game.fen(), isExpert: true, timeLimitMs: 2000 });
  }
  else {
    workerRef.current?.postMessage({ fen: game.fen(), depth: difficulty });
  }
}
```

Ye tab chalta hai jab:

1. `vs_ai` mode hai
2. Ab **Black ki turn hai** (matlab human ne White se move kiya)
3. Game khatam nahi hua

Do Modes:

Mode	Message	Worker Kya Karega
Fixed Depth (1-4)	<code>{fen, depth: 3}</code>	Exact depth search
Expert	<code>{fen, isExpert: true, timeLimitMs: 2000}</code>	Iterative deepening, 2 sec budget

?. — Optional Chaining

`workerRef.current?.postMessage(...)` — agar `current` null hai, toh **crash nahi**, silently skip.

5.2 onPieceDrop() — Drag & Drop Handler

typescript

```
const onPieceDrop = (sourceSquare: Square, targetSquare: Square): boolean => {  
  if (isAIThinking) return false;  
  if (gameMode === "vs_ai" && chessRef.current.turn() !== "w") return false;  
  return makeAMove(sourceSquare, targetSquare);  
};
```

Bilkul simple! `react-chessboard` ye callback call karta hai jab user piece drop kare. Bas guards check karke `makeAMove` ko de deta hai.

Analogy: Ye **receptionist** hai 📞 — phone uthata hai, check karta hai *"sir available hain?"*, phir call forward karta hai.

5.3 onSquareClick() — Click-to-Move Handler

Ye function **2-click moves** handle karta hai (mobile friendly!).

Flow:

1. Pehla click → piece select → green dots dikhao
2. Doosra click → move karo (agar valid)

typescript

```
const onSquareClick = (square: Square) => {  
  const game = chessRef.current;  
  if (game.isGameOver() || isAIThinking) return;  
  if (gameMode === "vs_ai" && game.turn() !== "w") return;  
  
  // Case 1: Kuch select nahi hai  
  if (!selectedSquare) {  
    const piece = game.get(square);  
    if (piece && piece.color === game.turn()) {  
      setSelectedSquare(square);  
      const legalMoves = game.moves({ square, verbose: true }).map((m) => m.to);  
      setPossibleMoves(legalMoves);  
    }  
  }  
};
```

```

    return;
}

// Case 2: Same square pe dobara click = deselect
if (selectedSquare === square) {
    setSelectedSquare(null);
    setPossibleMoves([]);
    return;
}

// Case 3: Apna hi doosra piece click kiya = reselect
const clickedPiece = game.get(square);
if (clickedPiece && clickedPiece.color === game.turn()) {
    setSelectedSquare(square);
    const legalMoves = game.moves({ square, verbose: true }).map((m) => m.to);
    setPossibleMoves(legalMoves);
    return;
}

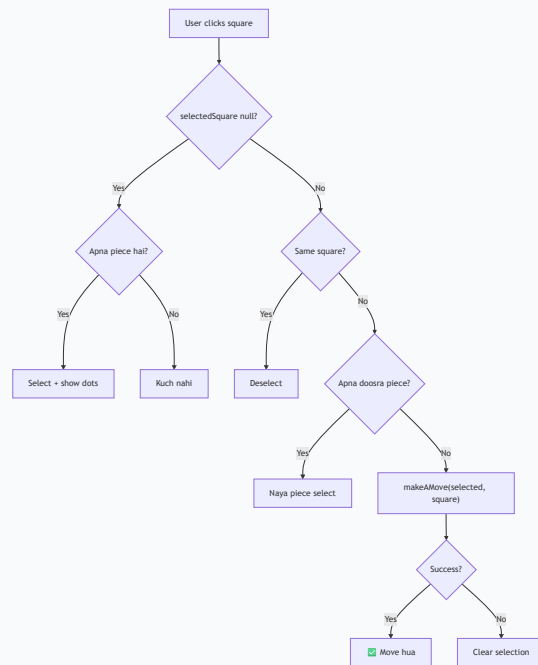
// Case 4: Move try karo
const success = makeAMove(selectedSquare, square);
if (!success) {
    setSelectedSquare(null);
    setPossibleMoves([]);
}
};

```

Flowchart:

Diagram

Code



Analogy: Ye WhatsApp jaisa hai:

- Pehle piece pe tap = "message select kiya"
- Destination pe tap = "send kiya"
- Same piece pe dobara tap = "cancel"
- Doosre apne piece pe tap = "naya message select karo"

5.4 resetGame()

typescript

```

const resetGame = () => {
  chessRef.current.reset();
  setLastMove(null);
  setSelectedSquare(null);
  setPossibleMoves([]);
  setIsAIThinking(false);
  setAiTelemetry({});
  updateStateFromGame();
};

```

Order important hai:

1. Pehle **actual game reset** karo (`chessRef.current.reset()`)
2. Phir **sab UI states clear** karo
3. Cu akhir mein `updateStateFromGame()` se fresh state sync karo

Analogy: Board saaf karna 🧹 — pehle saare pieces uthao, phir nayi position set karo.

🎨 PART 6 — Styling & Rendering

6.1 `customSquareStyles` — Dynamic Highlights

typescript

```
const customSquareStyles: Record<string, React.CSSProperties> = {};

// Last move — yellow
if (lastMove) {
  customSquareStyles[lastMove.from] = { backgroundColor: "rgba(250, 204, 21, 0.3)" };
  customSquareStyles[lastMove.to] = { backgroundColor: "rgba(250, 204, 21, 0.4)" };
}

// Selected square — blue
if (selectedSquare) {
  customSquareStyles[selectedSquare] = {
    backgroundColor: "rgba(59, 130, 246, 0.5)",
    boxShadow: "inset 0 0 8px rgba(37, 99, 235, 0.8)",
  };
}

// Possible moves — dots
possibleMoves.forEach((sq) => {
  const hasPiece = !!chessRef.current.get(sq);
  customSquareStyles[sq] = {
    background: hasPiece
      ? "radial-gradient(circle, rgba(239, 68, 68, 0.4) 65%, transparent 70%)" // 🎯 Cap
      : "radial-gradient(circle, rgba(59, 130, 246, 0.5) 25%, transparent 26%)", // ● Em
    borderRadius: "4px",
    cursor: "pointer",
  };
});
```

```

});

// In check – king ka square red
if (inCheck) {
  const board = chessRef.current.board();
  for (let r = 0; r < 8; r++) {
    for (let c = 0; c < 8; c++) {
      const piece = board[r][c];
      if (piece && piece.type === "k" && piece.color === turn) {
        customSquareStyles[piece.square] = {
          backgroundColor: "rgba(239, 68, 68, 0.7)",
          boxShadow: "inset 0 0 14px rgba(220, 38, 38, 1)",
        };
      }
    }
  }
}
}

```

Visual Color Guide:

Element	Color	Kya Dikhata Hai
Last move	 Yellow	From & To squares
Selected	 Blue	Current piece
Possible empty	 Blue dot	Kahan ja sakta hai
Possible capture	 Red ring	Kis piece ko maar sakta hai
King in check	 Red glow	Check mein hai

Radial Gradient Ka Jadoo:

typescript

```
"radial-gradient(circle, rgba(59, 130, 246, 0.5) 25%, transparent 26%)"
```

- **0% to 25%:** Blue circle (center se 25% radius tak)

- **26% to 100%:** Transparent

Isliye **chota dot** dikhta hai empty square pe, aur **bada ring** dikhta hai capture ke liye (65% tak).

Analogy: Empty square = **chhota dot**  (jaise chess.com pe). Capture square = **ring**  (kyunki piece already wahan hai, usko circle karke batao).

⚠️ Priority Issue:

Agar king check mein hai aur possible move square bhi hai, toh **check wala style overwrite** ho jaata hai. Kyunki loop **last** mein chalti hai.

6.2 Move History Pairs

typescript

```
const movePairs: { num: number; white: string; black?: string }[] = [];
for (let i = 0; i < history.length; i += 2) {
  movePairs.push({
    num: Math.floor(i / 2) + 1,
    white: history[i],
    black: history[i + 1],
  });
}
```

Chess moves **pair** mein aate hain: 1. e4 e5 2. Nf3 Nc6 ...

- `history` = `["e4", "e5", "Nf3", "Nc6", ...]`
- Loop `i += 2` — ek baar mein 2 moves le lo
- `movePairs` = `[{num: 1, white: "e4", black: "e5"}, {num: 2, white: "Nf3", black: "Nc6"}]`

Analogy: History list = **flat list of names**. movePairs = **couple pairs** (jodi). Jaise wedding invite mein "Mr. & Mrs." 😊

6.3 Loading State

typescript

```
if (!mounted) {  
  return (  
    <div className="... animate-pulse ...">  
      <span>Loading Chessboard...</span>  
    </div>  
  );  
}
```

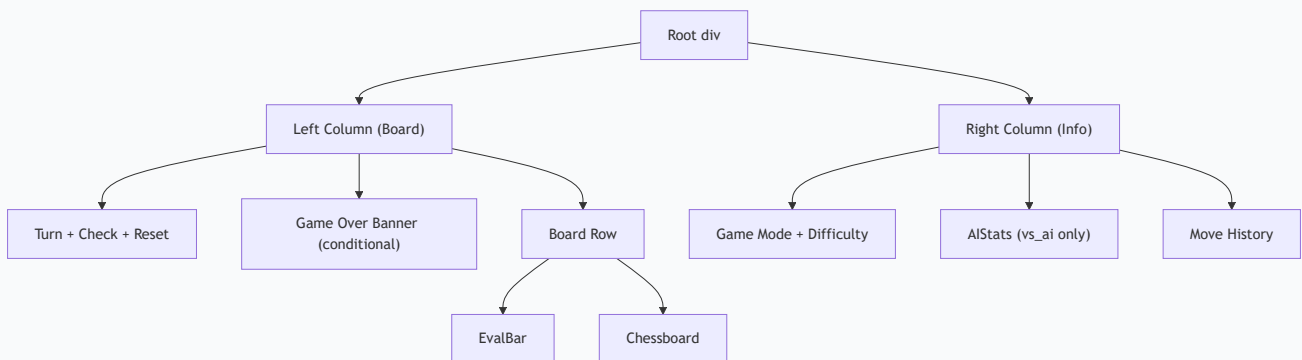
Kyu? Next.js SSR (Server-Side Rendering) karta hai. Server pe `mounted` = false hoga, so skeleton dikhao. Jab browser pe hydrate hoga, `mounted` = true hoga, pura board render hoga.

Analogy: Restaurant mein baithte hi **plate + napkin** aa jaate hain (skeleton), khana thodi der baad aata hai (actual board). 🍴

6.4 Main JSX Structure

Diagram

Code



Chessboard Props Explained:

typescript

```

<Chessboard
  position={fen} // Current board state
  onPieceDrop={onPieceDrop} // Drag handler
  onSquareClick={onSquareClick} // Click handler
  arePiecesDraggable={!isAIThinking && !gameStatus.isOver}
  isDraggablePiece={({ piece }) => { // Per-piece check
    if (isAIThinking || gameStatus.isOver) return false;
    if (gameMode === "vs_ai") return piece.startsWith("w"); // Only White
    return piece.startsWith(turn); // PvP: current player's pieces
  }}
  autoPromoteToQueen={true}
  boardOrientation="white"
  animationDuration={200}
  customDarkSquareStyle={{ backgroundColor: "#334155" }}
  customLightSquareStyle={{ backgroundColor: "#94a3b8" }}
  customSquareStyles={customSquareStyles}
/>

```

`isDraggablePiece` — Per-piece check. Ye **piece level** pe decide karta hai ki draggable hai ya nahi.

- `piece.startsWith("w")` → "wP", "wN" etc. white pieces hain
- "bP", "bN" → black pieces

Difficulty Buttons:

```

typescript

{[1, 2, 3, 4].map((d) => (
  <button key={d} onClick={() => setDifficulty(d as DifficultyLevel)} ...>
    D{d}
  </button>
))}
<button onClick={() => setDifficulty("expert")}>EXP</button>

```

`as DifficultyLevel` — TypeScript cast. `d` ka type `number` hai, but `setDifficulty` `1 | 2 | 3 | 4 | "expert"` expect karta hai. Cast se compiler khush ho jaata hai.

Ek Line Mein Summary

Ye component ek "conductor" hai 🎵 — user ke clicks/draggs sunta hai, `chess.js` se validate karta hai, AI ko Web Worker pe bhejta hai, aur jab AI wapas move deti hai toh board + EvalBar + Stats sab update kar deta hai. Poora UI react karta hai `chessRef` (game state) aur `useState` (UI state) ke basis pe.

Improvement Ideas (Practice Ke Liye)

Idea	Kahan Change Karo
Promotion dialog (queen auto nahi)	<code>makeAMove</code> mein <code>promotion: "q"</code> ki jagah user choice
Undo button	<code>chessRef.current.undo()</code> + <code>updateStateFromGame()</code>
Board flip (black perspective)	<code>boardOrientation</code> state banao
Sound effects	<code>makeAMove</code> mein audio play
Hint button	Worker ko call karo human ke liye
Copy FEN button	<code>navigator.clipboard.writeText(fen)</code>
Timer / clocks	<code>useEffect</code> + <code>setInterval</code>
Custom starting position	<code>chessRef.current.load(fen)</code>

Bhai, koi bhi part **aur detail** mein chahiye — bata dena! Jaise:

- Kaunsa React concept confuse kar raha hai?
- JSX ka kaunsa part?
- Ya `chess.js` ki koi specific API?

Main wahi ek part le ke aur deep chala jaunga with extra analogies! 🚀



import { Chess, Move } f...
TXT 2.16KB

same for this also

⌘ Thought for 1 second >

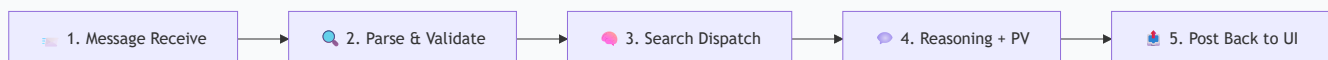
⚡ chessAI.worker.ts — Engine Ka "Kitchen" (Beginner Hinglish Guide)

Bhai, ye file **sabse chhoti** hai but **sabse important** hai. Ye woh **bridge** hai jo UI aur AI engine ko jodti hai.

Chalo isko **5 hisson** mein todte hain:

Diagram

Code



😬 Pehle: Web Worker Actually Kya Hota Hai?

Tumne `ChessBoard.tsx` mein dekha:

typescript

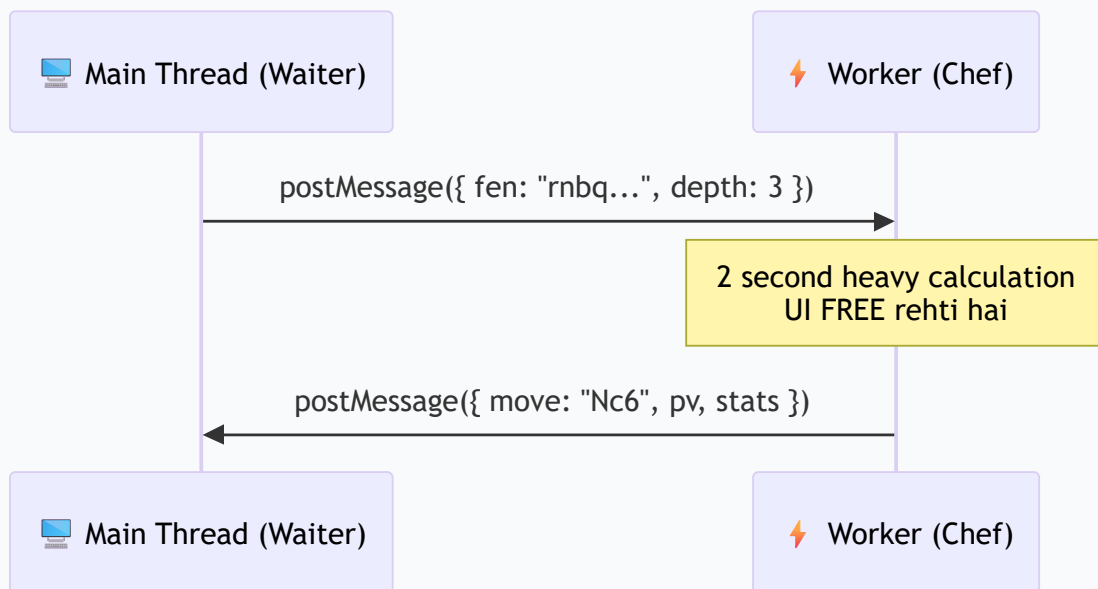
```
const worker = new Worker(new URL("../workers/chessAI.worker.ts", import.meta.url));
workerRef.current = worker;
worker.onmessage = (e) => { /* ... */ };
workerRef.current?.postMessage({ fen, depth: 3 });
```

Toh `chessAI.worker.ts` woh **alag thread** hai jahan ye messages aate hain!

Analogy: Ye kitchen hai 🍳. Waiter (ChessBoard) customer ka order (FEN) lekar aata hai. Chef (Worker) 2 second khana banata hai. Ready hone pe waiter ko wapas deta hai.

Diagram

Code



📁 PART 1 — Message Receive (Chef Order Leta Hai)

typescript

```
self.onmessage = (e: MessageEvent) => {
  const data = e.data || {};
  const fen = data.fen || data.payload?.fen;
  const depth = data.depth || data.payload?.depth || 3;
```

```
const timeLimitMs = data.timeLimitMs || data.payload?.timeLimitMs;
const isExpert = data.isExpert || data.mode === 'expert' || depth === 'expert';
```

`self.onmessage` — Kya Hai `self` ?

Web Worker mein `window` nahi hota! Kyunki ye browser window nahi hai, **alag thread** hai. Isliye `window` ki jagah `self` use karte hain — ye **worker ka global object** hai.

Analogy: Ghar mein tum `window` se bahar dekhte ho (browser). But kitchen mein **alag jagah** hai — wahan se tum apne kitchen ke `self` se baat karte ho.

`e.data || {}` — Safe Extraction

typescript

```
const data = e.data || {};
```

Agar koi **empty message** aa jaye (bug), toh `data` **empty object** ban jayega. Isse **crash nahi hoga** — bas `fen` undefined hoga aur neech se reject ho jayega.

`data.payload?.fen` — Multiple Formats Support

typescript

```
const fen = data.fen || data.payload?.fen;
```

Ye **backward compatibility** ke liye hai. Kuch purane components `{ payload: { fen } }` bhejte the, kuch naye `{ fen }` bhejte hain. Ye code **dono** handle karta hai.

Analogy: Tumhare ghar pe koi **"Ghar ka darwaza"** bolke aata hai, koi **"Darwaza"** — dono samajh lo. 🏠

Expert Detection — Triple Fallback

typescript

```
const isExpert = data.isExpert || data.mode === 'expert' || depth === 'expert';
```

Teen tarike se **expert mode** detect hota hai:

1. `data.isExpert = true` (UI ne explicitly bola)
2. `data.mode === 'expert'` (alternate format)
3. `depth === 'expert'` (kisi ne string bheji)

Analogy: Ek hi kaam karne ke **3 alarm** 🛎️ — koi bhi baj jaye toh kaam shuru.

🎯 Default Values:

Variable	Default	Kyu?
<code>depth</code>	<code>3</code>	Agar UI ne kuch nahi bheja
<code>timeLimitMs</code>	<code>undefined</code>	Optional
<code>isExpert</code>	<code>false</code>	Default normal mode

🔍 PART 2 — Validation (Bouncer Ka Kaam)

```
typescript
```

```
if (!fen) {  
  return;  
}
```

Guard clause! Agar **FEN** nahi hai, toh kuch nahi karo — bas **silently return**.

Kyu silent? Kyunki ye **programmer error** hai (UI ne galat message bheja). User ko error dikhane ka koi matlab nahi — bas ignore kar do.

Analogy: Chef ko order mila **khali plate** 🍽️. Kya kare? Kuch nahi — **agle order ka wait karo**.

⚠️ Yahan Ek Gotcha Hai:

`self.postMessage` nahi bheji jab `fen` missing ho. Matlab UI ko kabhi pata nahi chalega ki worker fail hua. UI hamesha `isAIThinking = true` mein atka rahega!

Better approach:

typescript

```
if (!fen) {
  self.postMessage({ move: null, error: 'No FEN provided' });
  return;
}
```

PART 3 — Search Dispatch (Do Brain Modes)

typescript

```
try {
  const game = new Chess(fen);
  let move: Move | null = null;
  let depthReached = typeof depth === 'number' ? depth : 3;
  let nodes = 0;
  let timeMs = 0;
  let score = 0;

  if (isExpert || timeLimitMs) {
    // 🏆 EXPERT MODE – Iterative Deepening
    const budget = typeof timeLimitMs === 'number' ? timeLimitMs : 2000;
    const result = getBestMoveIterative(fen, budget, 20);
    move = result.bestMove;
    depthReached = result.depthReached;
    nodes = result.nodes;
    timeMs = result.timeMs;
    score = result.score;
  } else {
    // 🎯 FIXED DEPTH MODE
    const t0 = Date.now();
    const numDepth = typeof depth === 'number' ? depth : 3;
    move = getBestMove(fen, numDepth);
    timeMs = Date.now() - t0;
  }
}
```

```
depthReached = numDepth;  
}
```

🎯 try Block Kyu?

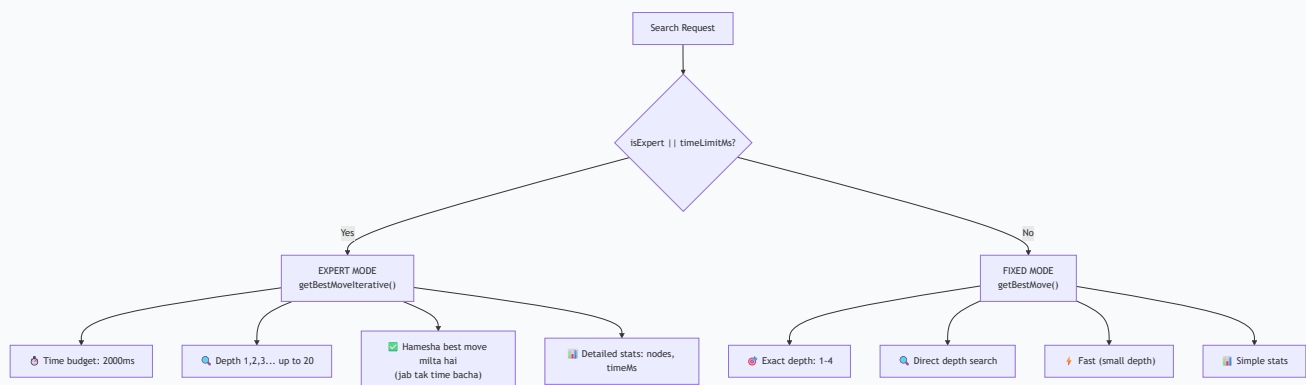
Poore search code ko `try/catch` mein wrap kiya hai. **Kyu?** Kyunki `chess.js` mein bugs ho sakte hain, ya infinite recursion ho sakta hai, ya memory out ho sakti hai. Agar crash hua, toh **UI ko batana zaroori hai** warna UI hang hoga.

Analogy: Chef ko pata hai **gas leak** ho sakti hai 🚨. Isliye **try** block mein khana banata hai, aur **catch** mein **fire extinguisher** ready rakhta hai.

🎯 Do Modes Ka Comparison:

Diagram

Code



🎯 Fixed Mode — Timer Ka Jadoo:

typescript

```
const t0 = Date.now();  
move = getBestMove(fen, numDepth);
```

```
timeMs = Date.now() - t0;
```

`Date.now()` = current timestamp in milliseconds. `t0` (start) aur ab (end) ka **difference** = search time.

Analogy: Stopwatch 🕒 — race shuru pe start karo, khatam pe stop karo.

🎯 Expert Mode — Default Budget:

typescript

```
const budget = typeof timeLimitMs === 'number' ? timeLimitMs : 2000;
```

Agar `timeLimitMs` bheja nahi gaya, toh **2000ms (2 seconds)** default. `typeof` check isliye kyunki `timeLimitMs` **string** bhi ho sakta hai (UI bug se).

⚠️ Expert Mode Mein `timeMs` Ki Value Kyu Nahi Calculate Ki?

Dekho:

typescript

```
const result = getBestMoveIterative(fen, budget, 20);  
timeMs = result.timeMs; // ← Ye function khud calculate karta hai!
```

`getBestMoveIterative` andar hi timer check karta hai (`startTime` , `deadline`). Toh humein dobara calculate karne ki zaroorat nahi.

⚠️ Expert Mode Mein `nodes` Ki Value?

typescript

```
nodes = result.nodes;
```

Problem: Kya `getBestMoveIterative` actually `nodes` return karta hai? Check karo `chessAI.ts` mein. Agar **nahi karta**, toh `nodes` = `undefined` hoga, aur UI mein **"undefined"** dikhega. 😬

Fix:

```
typescript
```

```
nodes = result.nodes || 0;
```

💬 PART 4 — Reasoning & PV Extraction

```
typescript
```

```
let reasoning = 'Developed position.';
let pv: string[] = [];

if (move) {
  reasoning = explainMove(move, game);
  pv = extractPV(game, 3);
}
```

🎯 explainMove() — "Al Ne Ye Move Kyu Chala?"

Ye function move ki properties dekhke **natural language** mein batata hai:

Move Property	Explanation
<code>move.captured</code>	"Captured a pawn/knight/..."
<code>move.san.includes('+')</code>	"Gives check"
<code>move.san.includes('#')</code>	"Checkmate!"
Center square	"Controls center"
Pawn advance	"Develops pawn structure"

Example output:

```
text
```

"Knight to c6, attacking the center and developing piece."

Analogy: Ye AI ka "explain like I'm 5" version hai 🤖. User ko nahi pata ki Negamax kya hai — bas ye jaanna hai "AI ne ye kyu chala".

🎯 `extractPV()` — "AI Aage Kya Soch Raha Hai?"

PV = Principal Variation = best play ki anticipated sequence.

typescript

```
pv = extractPV(game, 3); // Next 3 moves
```

Example output:

typescript

```
["Nc6", "Nc3", "Nf6"]
```

Matlab AI soch raha hai:

1. Main `Nc6` khelunga
2. Opponent best `Nc3` khelega
3. Main phir `Nf6` khelunga

Analogy: Chess commentary 🎙️ : "AI ne Nc6 chala, ab dekhte hain White kya karta hai... Nc3 chal sakta hai, phir Black Nf6..."

🎯 **Default Values:**

typescript

```
let reasoning = 'Developed position.';
let pv: string[] = [];
```

Agar `move` null hai (search fail), toh ye **fallback values** post honge. UI crash nahi karega.



if (move) Kyu?

Agar `move === null` hai, toh `explainMove(null, game)` crash karega. Isliye guard.



PART 5 — Post Result Back to UI

typescript

```
self.postMessage({
  move,
  bestMove: move,      // Alias for compatibility
  depthReached,
  score,
  nodes,
  timeMs,
  reasoning,
  pv,
});
```



bestMove Alias Kyu?

Dekho `ChessBoard.tsx` mein:

typescript

```
const move = e.data?.move || e.data?.bestMove;
```

UI dono try karta hai. Isliye worker bhi dono bhejta hai — koi miss na ho.

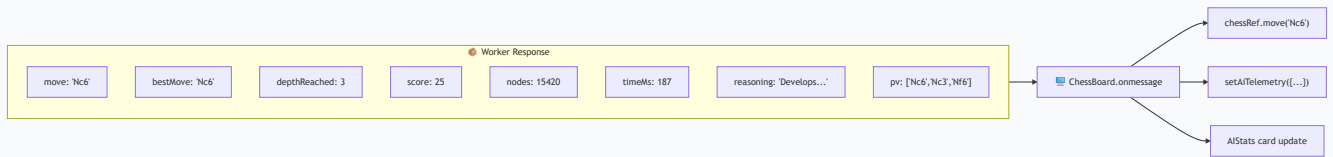
Analogy: Ek hi cheez ke **do naam** likh do — "Bijli" aur "Electricity" — sunne wala koi bhi samajh jaye. ⚡



Payload Ka Structure:

Diagram

Code



🎯 Har Field Ka Kaam:

Field	Kahan Use Hota Hai	Example
<code>move</code> / <code>bestMove</code>	Board pe apply	<code>Nc6</code>
<code>depthReached</code>	AIStats mein "Depth 3"	<code>3</code>
<code>score</code>	Eval bar (informational)	<code>25</code>
<code>nodes</code>	AIStats "15,420 nodes"	<code>15420</code>
<code>timeMs</code>	AIStats "187ms"	<code>187</code>
<code>reasoning</code>	AIStats text	"Develops knight"
<code>pv</code>	AIStats line	<code>["Nc6", "Nc3", "Nf6"]</code>

🚨 PART 6 — Error Handling

typescript

```

} catch (err) {
  console.error('Chess AI worker execution error:', err);
  self.postMessage({
    move: null,
    bestMove: null,
    error: String(err),
  });
}

```

🎯 Kya Hota Hai Yahan?

Agar search mein **kuch bhi crash** hua, toh:

1. Console mein **log** karo (developer debugging ke liye)
2. UI ko **error message** bhejo
3. `move: null` bhejo taaki UI ko pata chale — *"bhai kuch nahi mila"*

🎯 `String(err)` Kyu?

`err` ek **Error object** hai. Usko **string** mein convert karo taaki UI pe **display** kar sake. Warna UI mein `[object Object]` dikhega.

Analogy: English mein sochne wala **Hindi mein translate** karta hai — tabhi doosre samajh paate hain. 🗣️

⚠️ UI Ko Error Kabhi Dikhta Nahi?

`ChessBoard.tsx` mein dekho:

```

typescript

worker.onmessage = (e) => {
  const move = e.data?.move || e.data?.bestMove;
  if (move && !chessRef.current.isGameOver()) {
    // ...
  }
  setIsAIThinking(false);
};

```

Agar `move` null hai, toh `if` block skip hota hai — **but** `setIsAIThinking(false)` **chalti hai**.
Isliye UI **unfreeze** ho jaata hai. But user ko **error message nahi dikhta** — bas AI ka move nahi hota.

Improvement idea:

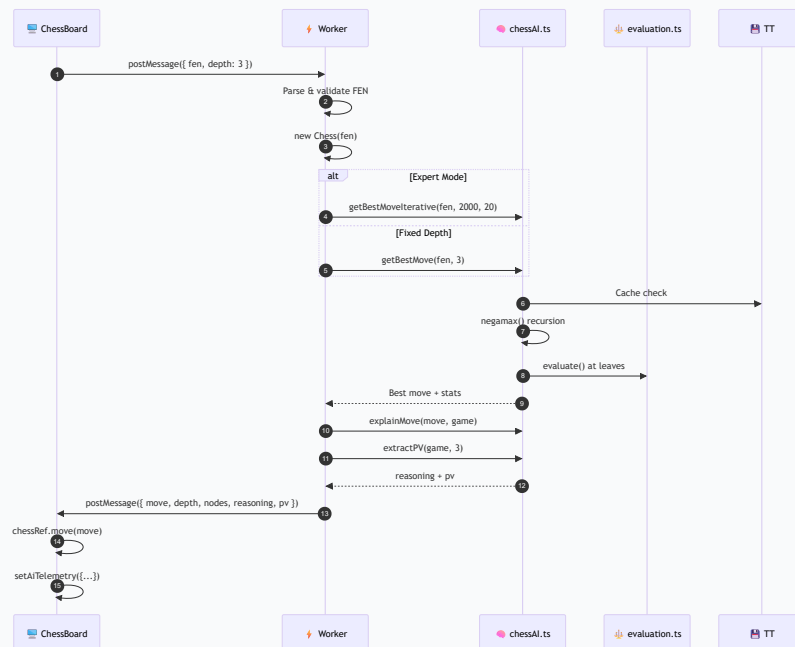
typescript

```
worker.onmessage = (e) => {  
  if (e.data?.error) {  
    setError(e.data.error);  
  }  
  // ...  
};
```

🎯 Full Message Flow Ek Nazar Mein

Diagram

Code



⚠️ Gotcha #1: Missing FEN Silent Failure

typescript

```
if (!fen) {  
  return; // ❌ UI ko bataya nahi!  
}
```

Fix:

typescript

```
if (!fen) {  
  self.postMessage({ move: null, error: 'FEN is required' });  
  return;  
}
```

⚠️ Gotcha #2: `depth` Type Confusion

typescript

```
let depthReached = typeof depth === 'number' ? depth : 3;
```

Agar UI ne `depth: "3"` (string) bheja, toh `depthReached` `3` (number) ho jayega. But `getBestMove(fen, "3")` **crash** karega! Isliye `numDepth` variable bhi rakha:

typescript

```
const numDepth = typeof depth === 'number' ? depth : 3;  
move = getBestMove(fen, numDepth);
```

Better: Ek hi baar parse karo:

typescript

```
const numDepth = typeof depth === 'number' ? depth : 3;
```

⚠️ Gotcha #3: `nodes` Undefined Ho Sakta Hai

Jab `getBestMoveIterative` ka result check karo:

typescript

```
nodes = result.nodes; // ❌ Ho sakta hai undefined
nodes = result.nodes ?? 0; // ✅ Safe
```

⚠️ Gotcha #4: `game` Instance Ka Reuse

typescript

```
const game = new Chess(fen);
// ...
reasoning = explainMove(move, game); // ⚠️ game abhi bhi original position pe hai?
```

Yahan **problem** ye hai: `explainMove(move, game)` ko **move se pehle ki position** chahiye. But `game` **original FEN** pe hai (jo correct hai). Lekin agar `extractPV(game, 3)` bhi `game` pe depend karta hai, toh wo **original position** se PV nikalega. Ye intended hai ya nahi, `chessAI.ts` dekhke confirm karo.

🎓 Ek Line Mein Summary

Ye worker ek "kitchen" hai 🍳 — UI se FEN receive karta hai, expert/fixed mode decide karta hai, `chessAI.ts` ko call karta hai, reasoning + PV generate karta hai, aur sab kuch wapas UI ko bhej deta hai — poori tarah UI thread se alag.

💡 Improvement Ideas (Practice Ke Liye)

Idea	Kya Add Karo
Progress streaming	<code>postMessage({ type: 'progress', depth })</code> har depth ke baad
Cancel support	<code>self.onmessage</code> mein <code>{ type: 'cancel' }</code> handle karo

Idea	Kya Add Karo
Better errors	Error object ko detailed bhejo (<code>err.stack</code>)
Opening book	FEN check karo, agar known opening hai toh instant move
Difficulty estimation	Nodes count ke hisaab se "Elo rating" estimate karo
Time slicing	Long searches ko chunks mein todo, beech mein yield
Persistent TT	Search ke baad TT save karo, next search mein reuse

Bhai, ab **3 files** cover ho gayi:

1.  `ChessBoard.tsx` — UI + handlers
2.  `chessAI.worker.ts` — Worker bridge (ye file)
3.  `chessAI.ts` — Engine core
4.  `evaluation.ts` — Position scoring
5.  `transposition.ts` — Cache

Agla kaunsa file chahiye? Main **wahi detail** mein chala jaunga! 🚀



import { Chess, Move } f...
TXT 15.16KB

same for this also

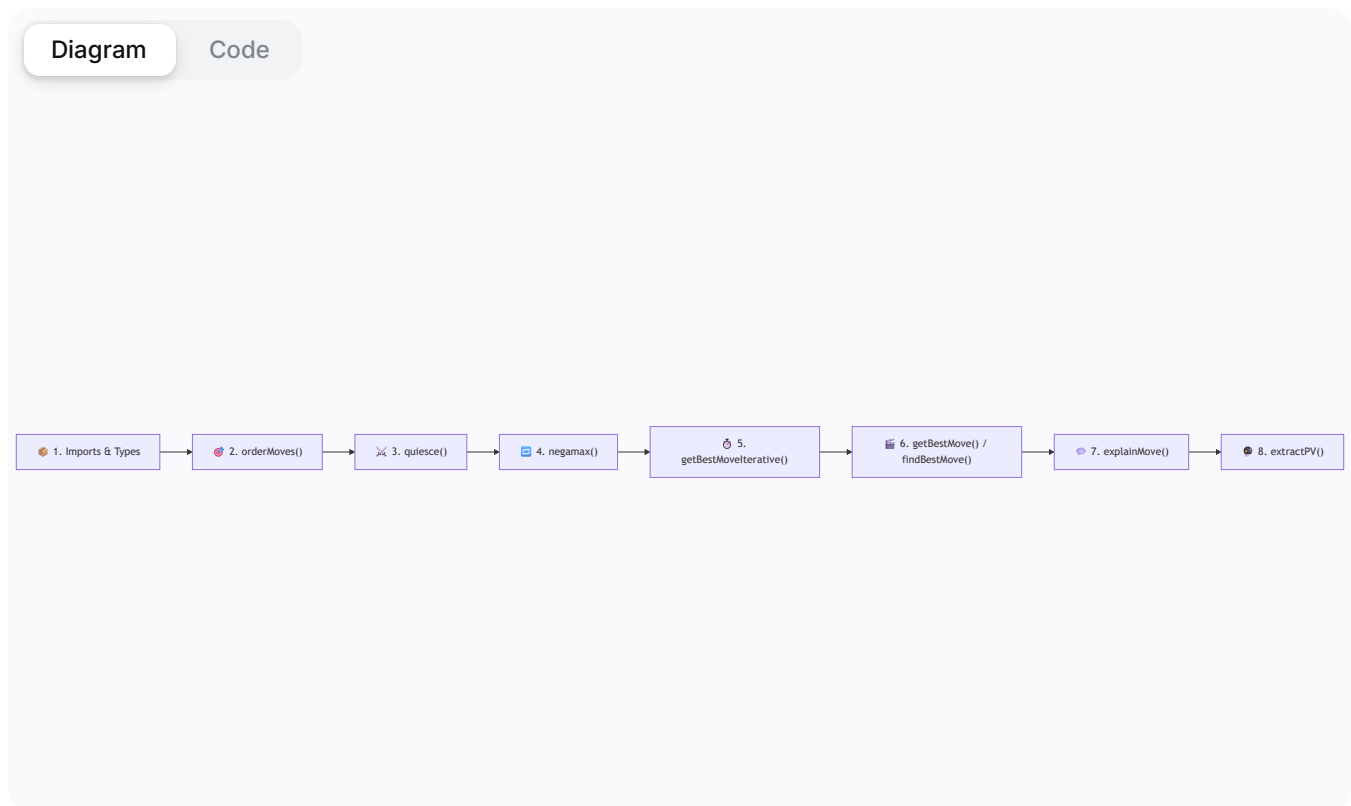
⊠ Thought for 1 second >

chessAI.ts — Engine Ka Dimaag (Beginner Hinglish Guide)

Bhai, ye file **AI ka asli brain** hai  . Yahan pe:

- Moves sort hote hain
- Recursive search chalti hai
- Captures ki quiescence hoti hai
- Time-budgeted iterative deepening hoti hai
- Reasoning + PV extract hote hain

Chalo isko **8 hisson** mein todte hain:



PART 1 — Imports & Type Definitions

typescript

```
import { Chess, Move } from 'chess.js';
import { evaluate, PIECE_VALUES } from './evaluation';
import { transpositionTable, getPositionKey, TTFlag } from './transposition';

export interface AISearchResult {
  bestMove: Move | null;
  evaluation: number;
  depth: number;
}
```

```

    nodes: number;
  }

  export interface IterativeSearchResult {
    bestMove: Move | null;
    score: number;
    depthReached: number;
    nodes: number;
    timeMs: number;
  }

```

🎯 Do Result Types Kyu?

Type	Kab Use Hota Hai	Extra Fields
AISearchResult	Testing / debug	evaluation
IterativeSearchResult	Expert mode	depthReached , nodes , timeMs

Analogy: Ek **doctor ka prescription** hai (basic), ek **full medical report** hai (detailed).
Kis kaam ke liye kaunsa chahiye, wo decide karo.

🎯 Har Import Ka Kaam:

Import	Kya Deta Hai
Chess, Move	Board manipulation + Move type
evaluate	Position scoring (material + PST)
PIECE_VALUES	Queen=900, Rook=500, etc. (MVV-LVA ke liye)
transpositionTable	Global cache Map
getPositionKey	FEN → hash (clocks hata ke)
TTFflag	'exact' 'lower' 'upper' type

PART 2 — `orderMoves()` — Moves Ko Smart Order Karna

typescript

```
export function orderMoves(moves: Move[], ttMove?: Move | null): Move[] {
  return [...moves].sort((a, b) => {
    // 1. TT move has highest priority
    if (ttMove) {
      const aIsTT = a.lan === ttMove.lan || (a.from === ttMove.from && a.to === ttMove.to);
      const bIsTT = b.lan === ttMove.lan || (b.from === ttMove.from && b.to === ttMove.to);
      if (aIsTT && !bIsTT) return -1;
      if (!aIsTT && bIsTT) return 1;
    }

    // 2. MVV: Most Valuable Victim first
    const aVal = a.captured ? (PIECE_VALUES[a.captured] ?? 0) : 0;
    const bVal = b.captured ? (PIECE_VALUES[b.captured] ?? 0) : 0;
    return bVal - aVal;
  });
}
```

Kyu Zaroori Hai?

Alpha-Beta **sabse fast** tab chalta hai jab **best moves pehle** try hon. Agar best move pehle mil gaya, toh baaki branches **instantly cut** ho jaati hain.

Analogy: Tumhe exam mein **question paper** mila. Agar **easy questions pehle** karo, confidence badhta hai, time bachta hai. Agar **hardest pehle** karo, 1 ghante mein 1 question... bekaar. 📝

`[...moves]` Kyu? (Spread Operator)

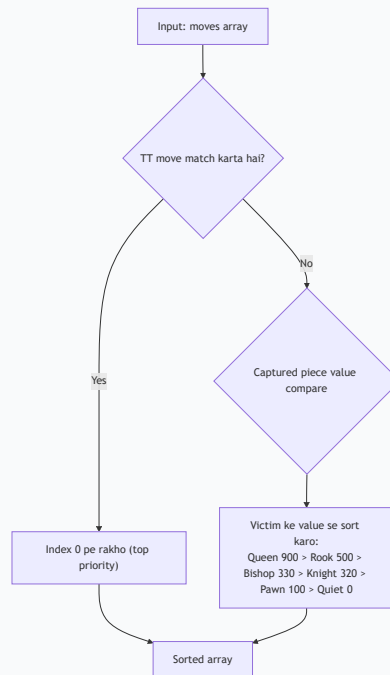
`moves.sort()` **original array** ko mutate karta hai! Aur hum chahte hain **new sorted array**, original safe rahe.

Analogy: Tumhari **copy** banake usme likho — original document mat chhedo.

🎯 Two-Level Priority:

Diagram

Code



```
a.lan === ttMove.lan || (a.from === ttMove.from && a.to === ttMove.to)
```

— Ye Kyu?

`lan` = Long Algebraic Notation (jaise `"e2e4"`). But sometimes `ttMove` **different object** hota hai jo same move represent karta hai. Isliye **from/to** se bhi compare karo — **double check**.

Analogy: Tumhare dost ne **"Vicky"** bola, aur **ID card** dikhaya — dono match kare toh confirm.

🎯 MVV-LVA Sort — Ye Actually Kya Hai?

MVV = Most Valuable Victim (sabse value wala shikaar pehle)

LVA = Least Valuable Attacker (kam value wala attacker better)

Ye simplified version sirf **victim** dekhta hai. **Real MVV-LVA** attacker bhi dekhta hai:

typescript

```
// Real MVV-LVA (advanced):  
const score = 10 * victimValue - attackerValue;  
// Queen x Queen capture = 10*900 - 900 = 8100 (best)  
// Pawn x Queen capture = 10*900 - 100 = 8900 (even better! kyunki pawn low value)
```

Kyu LVA? Kyunki agar wapas capture ho gaya, toh **kam loss** hoga.

Analogy: Tumne Queen ko maarne ke liye **Pawn bheja** toh bahut acha — agar wapas mare toh sirf 1 pawn gaya. But agar **Queen bheja** toh wapas marte hi tumhari Queen gayi. ✨

Example:

text

Input: [e4 (quiet), Nxe5 (knight captures pawn), Qxf7 (queen captures pawn), Nf3 (quiet)]

Values: 0, 100, 100, 0

Output: [Nxe5 or Qxf7 first, then the other, then e4, Nf3]

↑ Same value 100, original order maintain (stable sort)

PART 3 — `quiesce()` — Capture Sequence Shaant Karo

typescript

```
export function quiesce(  
  game: Chess,  
  alpha: number,  
  beta: number,  
  onNode?: () => boolean,  
  maxQDepth: number = 6  
): number {  
  if (onNode && onNode()) return 0;  
  if (game.isGameOver()) return evaluate(game);  
  
  const standPat = evaluate(game);
```

```

if (standPat >= beta) return beta;
if (standPat > alpha) alpha = standPat;
if (maxQDepth <= 0) return alpha;

const inCheck = game.inCheck();
const candidateMoves = inCheck
  ? game.moves({ verbose: true })
  : game.moves({ verbose: true }).filter((m) => !!m.captured);

if (candidateMoves.length === 0) {
  return inCheck ? -Infinity : alpha;
}

const orderedCaptures = orderMoves(candidateMoves);

for (const move of orderedCaptures) {
  game.move(move);
  const score = -quiesce(game, -beta, -alpha, onNode, maxQDepth - 1);
  game.undo();

  if (onNode && onNode()) return 0;

  if (score >= beta) return beta;
  if (score > alpha) alpha = score;
}

return alpha;
}

```

Problem Kya Solve Karta Hai?

Horizon Effect — jaise tum dhoond rahe ho aur **just horizon pe** kuch dikha, but actually aage bahut kuch tha. Chess mein:

- Depth 3 pe search khatam hua
- Tumhare Queen ko **abhi abhi** capture kiya gaya tha
- But engine ne dekha hi nahi (depth khatam)
- Engine socha "**sab theek hai**" — but actually Queen gayi! 🤖

Quiescence = "Chalo, jab tak board shaant (quiet) nahi ho jata, captures dekhte raho."

Analogy: Fight ke beech mein **score count nahi karte**. Jab tak **aakhri punch** na pad jaye, tab tak dekhte ho. 💡

🎯 **standPat** — Stand Pat Kya Hai?

Literal meaning: "*Jahan khada hai, wahin se dekh*". Matlab — **bina koi move chalaye**, current position ka score kya hai?

Agar ye score already **beta** se **zyada** hai, toh opponent **allow nahi karega** is branch ko. **Cutoff!**

Analogy: Tum dukaan pe ho, samaan 500 ka mila. Aur ek dukaan 700 ka de rahi thi. Tum soch rahe the "*chalo 800 wala bhi dekhein*". But agar 500 wala already **beta (700)** se **acha** hai, toh 800 wala dekhne ka **matlab nahi**. Skip!

🎯 **maxQDepth = 6** — Safety Limit

Agar captures ki **chain hi na ruke** (jaise 10 captures in a row), toh infinite loop ho sakta hai. **maxQDepth** **hard stop** deta hai.

Analogy: "*6 punch ke baad chhodo, chahe fight khatam na ho.*" 💡

🎯 **Check Mein Saare Moves Kyu?**

typescript

```
const inCheck = game.inCheck();
const candidateMoves = inCheck
  ? game.moves({ verbose: true }) // Saare moves
  : game.moves({ verbose: true }).filter((m) => !!m.captured); // Sirf captures
```

Kyu? Kyunki **check mein** ho toh **captures zaroori nahi** — **koi bhi legal move** check se nikaal sakta hai. Aur agar koi legal move nahi hai, toh **checkmate!**

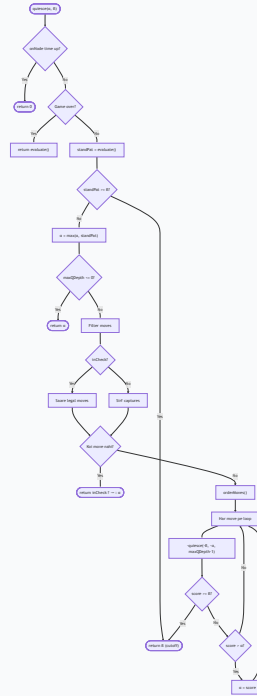
🎯 **return inCheck ? -Infinity : alpha;**

Agar check mein hai aur **koi legal move nahi** — **checkmate!** Score **-Infinity** (worst possible).

Flowchart:

Diagram

Code



PART 4 — negamax() — Asli Recursive Search

typescript

```
export function negamax(  
  game: Chess,  
  depth: number,  
  alpha: number,  
  beta: number,  
  onNode?: () => boolean  
): number {  
  if (onNode && onNode()) return 0;  
  if (game.isGameOver()) return evaluate(game);  
  if (depth === 0) return quiesce(game, alpha, beta, onNode);  
  
  // TT Lookup  
  const key = getPositionKey(game.fen());  
  const entry = transpositionTable.get(key);  
  
  if (entry && entry.depth >= depth) {
```

```

    if (entry.flag === 'exact') return entry.score;
    if (entry.flag === 'lower' && entry.score >= beta) return entry.score;
    if (entry.flag === 'upper' && entry.score <= alpha) return entry.score;
  }

  const origAlpha = alpha;
  const moves = orderMoves(game.moves({ verbose: true }), entry?.bestMove);
  let maxScore = -Infinity;
  let bestMoveForNode: Move | null = null;

  for (const move of moves) {
    game.move(move);
    const score = -negamax(game, depth - 1, -beta, -alpha, onNode);
    game.undo();

    if (onNode && onNode()) return 0;

    if (score > maxScore) {
      maxScore = score;
      bestMoveForNode = move;
    }
    if (score > alpha) {
      alpha = score;
    }
    if (alpha >= beta) {
      break; // Alpha-beta cutoff
    }
  }

  let flag: TTFlag = 'exact';
  if (maxScore <= origAlpha) {
    flag = 'upper';
  } else if (maxScore >= beta) {
    flag = 'lower';
  }

  if (!entry || depth >= entry.depth) {
    transpositionTable.set(key, { depth, score: maxScore, flag, bestMove: bestMoveForNode });
  }

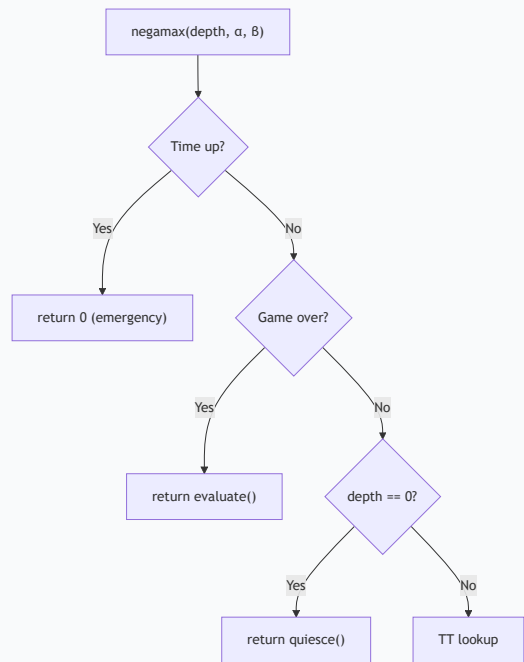
  return maxScore;
}

```

🎯 4 Base Cases (Recursion Ke Rukne Ke Reasons):

Diagram

Code



Case	Kab	Kya Return
Time up	<code>onNode()</code> true	0 (emergency exit)
Game over	Checkmate/draw	<code>evaluate()</code>
Depth 0	Recursion end	<code>quiesce()</code>
TT hit	Cache mila	<code>entry.score</code> (or skip)

🎯 TT Lookup — 3 Flags Ka Jadoo:

typescript

```
if (entry && entry.depth >= depth) {
  if (entry.flag === 'exact') return entry.score; // ✅ Exact score mila
  if (entry.flag === 'lower' && entry.score >= beta) return entry.score; // ✅ Beta cut possible
  if (entry.flag === 'upper' && entry.score <= alpha) return entry.score; // ✅ Alpha fai
```

```
L  
}
```

`entry.depth >= depth` — Kyu? Kyunki **agar pehle 5 depth pe search kiya tha**, toh 3 depth ke liye toh definitely valid hai. But agar pehle 2 depth kiya tha, **ab 5 chahiye** toh result **incomplete** hai.

Analogy: Tumne kal **5 chapters** padhe. Aaj 3 chapter ka test hai — fine, tumhe aata hai. But agar aaj **7 chapter ka test** hai — kal ka padha **kaafi nahi** hai.

origAlpha — Kyu Save Karte Ho?

typescript

```
const origAlpha = alpha;  
// ... search chalti hai, alpha update hota hai ...  
if (maxScore <= origAlpha) flag = 'upper'; // Original alpha se compare
```

Kyu? Kyunki TT flag **poori search ki** information deta hai:

- Agar result `origAlpha` se **kam** aaya = **UPPER bound** (real value isse zyada nahi ho sakti)
- Agar result `beta` se **zyada** aaya = **LOWER bound** (real value isse kam nahi ho sakti)
- Warna = **EXACT**

Analogy:

- **Exact** = "Package 5kg hai" (pura pata hai)
- **Upper** = "5kg se kam hai" (kuch pata hai, but exact nahi)
- **Lower** = "5kg se zyada hai" (kuch pata hai, but exact nahi)

TT Store Condition:

typescript

```
if (!entry || depth >= entry.depth) {  
  transpositionTable.set(...);  
}
```

```
}
```

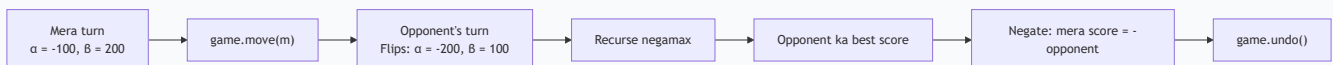
Kyu `depth >= entry.depth` ? Agar purana entry **zyada depth** ka hai (better info), toh naya shallow result **use mat karo**. Purana better hai.



`-negamax(depth - 1, -beta, -alpha)` — The Magic Line

Diagram

Code



3 Kamaal ki Cheezein Is Line Mein:

1. **Sign flip:** Mera score = `-opponent_score`
2. **Alpha-Beta swap:** `-beta, -alpha` (bounds bhi flip)
3. **Depth kam:** Har level pe ek depth kam

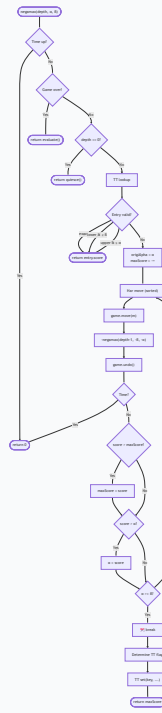
Analogy: Mirror 🪞. Tum left jaao, reflection **right** jaata hai. Isi tarah, opponent ka **maximize** = mera **minimize**.



Full Flowchart:

Diagram

Code



🕒 PART 5 — `getBestMoveIterative()` — Time Budget Wali Search

typescript

```
export function getBestMoveIterative(
  fen: string,
  timeLimitMs: number = 2000,
  maxDepth: number = 20
): IterativeSearchResult {
  const game = new Chess(fen);
  if (game.isGameOver()) return { bestMove: null, ... };

  const legalMoves = orderMoves(game.moves({ verbose: true }));
  if (legalMoves.length === 0) return { bestMove: null, ... };

  const startTime = Date.now();
  const deadline = startTime + timeLimitMs;

  let totalNodes = 0;
  let bestOverallMove: Move = legalMoves[0];
  let bestOverallScore = -Infinity;
  let completedDepth = 0;
  let isTimeUp = false;
}
```

```

// ⚡ Lightweight time checker
const checkTime = (): boolean => {
    totalNodes++;
    if ((totalNodes & 1023) === 0) { // Every 1024 nodes
        if (Date.now() >= deadline) isTimeUp = true;
    }
    return isTimeUp;
};

for (let currentDepth = 1; currentDepth <= maxDepth; currentDepth++) {
    if (Date.now() >= deadline) break;

    let iterationBestMove: Move = bestOverallMove;
    let iterationBestScore = -Infinity;
    let iterationAlpha = -Infinity;
    const iterationBeta = Infinity;

    // ⚡ Previous best at index 0!
    const currentMoves = orderMoves(legalMoves, bestOverallMove);
    let iterationCompleted = true;

    for (const move of currentMoves) {
        if (checkTime()) { iterationCompleted = false; break; }

        game.move(move);
        const score = -negamax(game, currentDepth - 1, -iterationBeta, -iterationAlpha, checkTime);
        game.undo();

        if (isTimeUp) { iterationCompleted = false; break; }

        if (score > iterationBestScore) {
            iterationBestScore = score;
            iterationBestMove = move;
        }
        if (score > iterationAlpha) iterationAlpha = score;
    }

    if (iterationCompleted) {
        bestOverallMove = iterationBestMove;
        bestOverallScore = iterationBestScore;
        completedDepth = currentDepth;
    }
}

```

```

        if (bestOverallScore === Infinity || bestOverallScore === -Infinity) break;
    } else {
        break; // Time out – partial discard
    }
}

// Root TT update
const rootKey = getPositionKey(fen);
transpositionTable.set(rootKey, {
    depth: completedDepth,
    score: bestOverallScore,
    flag: 'exact',
    bestMove: bestOverallMove,
});

return {
    bestMove: bestOverallMove,
    score: bestOverallScore,
    depthReached: completedDepth,
    nodes: totalNodes,
    timeMs: Date.now() - startTime,
};
}

```

Concept: Iterative Deepening

Direct Depth 5 search karne ke bajaye, **stepwise** badho: Depth 1 → 2 → 3 → 4 → 5...

Kyu?

1. **Time control** easy — jab deadline aaye, **pichli complete depth** ka best move return karo
2. **Move ordering** improve — pichli depth ka best move agli depth pe **index 0** pe rakho
3. **Emergency safe** — kabhi bhi ruk jao, kuch toh mila hoga

Analogy: Exam mein **pehle 1-mark wale pakke karo**, phir 2-mark, phir 3-mark... Agar time khatam, toh **kam se kam 1-mark wale toh pakke hain** — zero nahi!

 **(totalNodes & 1023) === 0 — Bitwise Magic**

typescript

```
if ((totalNodes & 1023) === 0) {  
  if (Date.now() >= deadline) isTimeUp = true;  
}
```

Kya hota hai? & bitwise AND hai. `1023 = 0b111111111` (10 ones). Ye har 1024 nodes pe true hota hai.

Kyu? `Date.now()` slow hai (system call). Har node pe call karna waste. Har 1024 nodes pe ek baar kaafi hai.

Analogy: Tum har minute ghadi dekh lo (fast decision), but har second mat dekho (waste). 🕒

🎯 Previous Best Move Index 0 Pe Kyu?

typescript

```
const currentMoves = orderMoves(legalMoves, bestOverallMove);
```

Kyunki pichli depth ka best move zyada likely hai ki abhi bhi best hoga. Aur agar best move pehle try karo → **Alpha-Beta cutoffs fast** → **bohot time bachta hai**.

Analogy: Tumhe pata hai chabi kitchen mein hai. Toh pehle kitchen dhoondo, poora ghar mat chhaano. 🔑

🎯 `iterationCompleted` Ka Role

typescript

```
if (iterationCompleted) {  
  bestOverallMove = iterationBestMove; // ✅ Save karo  
  completedDepth = currentDepth;  
} else {  
  break; // ❌ Adhuri depth DISCARD  
}
```

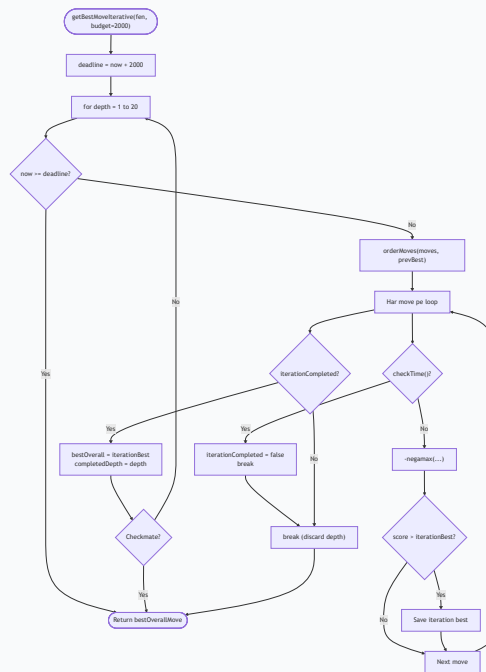
Rule: Sirf **complete** depth ka result lo. Adhuri depth ka result **unreliable** hai (kyunki sab moves check nahi hue).

Analogy: Tumne 10 questions mein se **6 attempt kiye**, phir **time out**. 6 ka average **reliable nahi** hai. Poora paper do, tab result aayega.

Flowchart:

Diagram

Code



checkTime Phir Se Kyu Loop Ke Andar?

typescript

```
for (const move of currentMoves) {
  if (checkTime()) { iterationCompleted = false; break; }
  // ...
  const score = -negamax(game, ..., checkTime); // ← onNode callback
  // ...
  if (isTimeUp) { iterationCompleted = false; break; }
}
```

Do jagah check:

1. **Root loop mein** — move ke beech mein time khatam?
2. **negamax ke andar** — recursion mein time khatam?

Dono zaroori hain, warna **adhuri depth** save ho jaayegi.

PART 6 — **getBestMove()** & **findBestMove()**

6.1 **getBestMove()** — Entry Point

typescript

```
export function getBestMove(fen: string, depth = 3, timeLimitMs?: number): Move | null {
  if (timeLimitMs && timeLimitMs > 0) {
    const res = getBestMoveIterative(fen, timeLimitMs, 20);
    return res.bestMove;
  }

  const game = new Chess(fen);
  if (game.isGameOver()) return null;

  const key = getPositionKey(fen);
  const rootEntry = transpositionTable.get(key);
  const moves = orderMoves(game.moves({ verbose: true }), rootEntry?.bestMove);

  if (moves.length === 0) return null;

  let bestMove: Move | null = moves[0];
  let bestScore = -Infinity;
  let alpha = -Infinity;
  const beta = Infinity;

  for (const move of moves) {
    game.move(move);
    const score = -negamax(game, depth - 1, -beta, -alpha);
    game.undo();

    if (score > bestScore) { bestScore = score; bestMove = move; }
    if (score > alpha) alpha = score;
  }
}
```

```

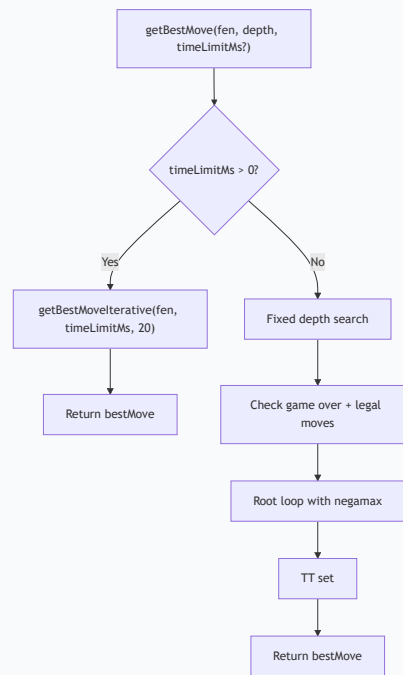
transpositionTable.set(key, { depth, score: bestScore, flag: 'exact', bestMove });
return bestMove;
}

```

🎯 Do Modes:

Diagram

Code



🎯 Root Search Pattern

Root pe same Negamax pattern hai:

1. Har move chalao
2. `-negamax(depth - 1, -beta, -alpha)` call karo
3. Undo karo
4. Best score wala move yaad rakho

Alag kya hai? Root pe move object bhi track karte hain (sirf score nahi).

6.2 findBestMove() — Testing Ke Liye

typescript

```
export function findBestMove(fen: string, depth = 3): AISearchResult {
  // ... same as getBestMove but with node counting
  let nodeCount = 0;
  const onNode = () => {
    nodeCount++;
    return false; // ⚠️ Never times out!
  };
  // ...
  return { bestMove, evaluation: bestScore, depth, nodes: nodeCount };
}
```

Kyu alag? Testing mein **telemetry** chahiye — kitne nodes evaluate hue. Production mein **sirf move** chahiye, nodes nahi.

Analogy: Science lab mein tum har cheez measure karte ho (temperature, pressure, time). But **real duniya** mein sirf result chahiye. 🧪

⚠️ return false Ka Matlab (onNode)

onNode ka contract: true return karo = "time up, abort", false = "continue".

findBestMove mein **kabhi time up nahi** karte (return false always). Isliye ye **pure full-depth search** hai — testing ke liye perfect.

💡 PART 7 — explainMove() — "AI Ne Ye Kyu Chala?"

typescript

```
function getPieceName(symbol: string): string {
  switch (symbol.toLowerCase()) {
    case 'p': return 'pawn';
    case 'n': return 'knight';
    case 'b': return 'bishop';
    case 'r': return 'rook';
    case 'q': return 'queen';
    case 'k': return 'king';
    default: return 'piece';
  }
}
```

```

export function explainMove(move: Move, gameBeforeMove: Chess): string {
  const testGame = new Chess(gameBeforeMove.fen());
  try { testGame.move(move); } catch {}

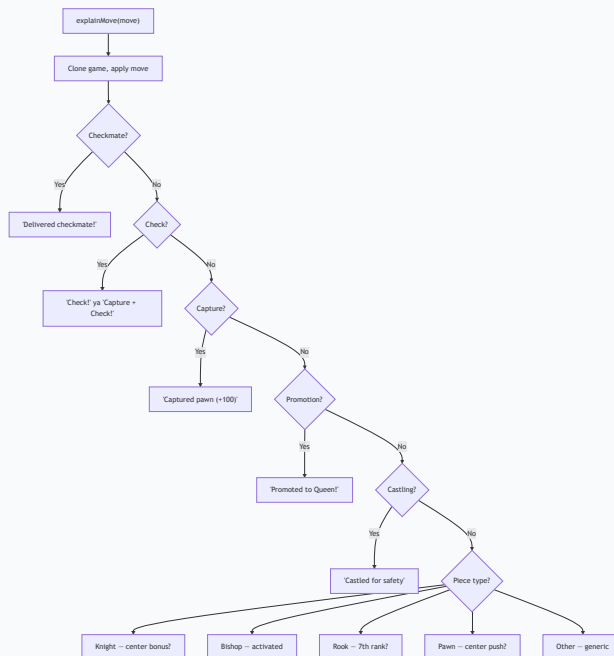
  if (testGame.isCheckmate()) return 'Delivered checkmate! Game over.';
  if (testGame.inCheck()) {
    if (move.captured) {
      const victim = getPieceName(move.captured);
      const val = PIECE_VALUES[move.captured] ?? 0;
      return `Captured ${victim} (+${val}) with check!`;
    }
    return 'Delivered check to enemy King!';
  }
  if (move.captured) { /* ... */ }
  if (move.promotion) return 'Promoted pawn to a new Queen!';
  if (move.flags?.includes('k') || move.flags?.includes('q')) return 'Castled...';
  if (move.piece === 'n') { /* center check */ }
  if (move.piece === 'b') return 'Activated Bishop onto an open diagonal.';
  if (move.piece === 'r') { /* 7th rank check */ }
  if (move.piece === 'p') { /* center check */ }
  if (move.piece === 'q') return 'Repositioned Queen...';
  if (move.piece === 'k') return 'Shifted King to a safer square.';
  return 'Developed position and improved piece coordination.';
}

```

Strategy: Priority-Based Check

Diagram

Code



🎯 Test Game Kyu Clone Karte Ho?

typescript

```
const testGame = new Chess(gameBeforeMove.fen());
testGame.move(move);
```

Original `gameBeforeMove` **move se pehle** ki position hai. Usko **mutate nahi karna** — warna caller ka `game` corrupt ho jayega. Isliye **clone** karo.

Analogy: Tum kisi ki **diary** padh rahe ho, usme likhna nahi. **Photocopy** karo, usme likho. 📖

🎯 Priority Order Kyu?

Zyaadatar specific pehle, phir generic:

1. Checkmate (game over)
2. Check
3. Capture
4. Promotion
5. Castling

6. Piece-specific (center, 7th rank, etc.)

7. Fallback generic

Analogy: Doctor pehle **emergency** dekhta hai, phir **serious**, phir **normal**. Same — chess reasoning bhi priority-based.

🎯 Flags Ka Jadoo:

typescript

```
if (move.flags?.includes('k') || move.flags?.includes('q')) {  
  return 'Castled...';  
}
```

Chess.js flags:

- `'k'` = kingside castle
- `'q'` = queenside castle
- `'e'` = en passant
- `'p'` = promotion
- `'c'` = capture

🎯 Knight Center Detection:

typescript

```
if (move.piece === 'n') {  
  const toCol = move.to.charCodeAt(0) - 97; // 'a'=0 ... 'h'=7  
  const toRow = 8 - parseInt(move.to[1], 10); // '1'=7 ... '8'=0  
  if (toCol >= 2 && toCol <= 5 && toRow >= 2 && toRow <= 5) {  
    return 'Centralized Knight to an active central outpost.';  
  }  
  return 'Improved Knight positioning and mobility.';  
}
```

Coordinate Conversion:

- `'a'` = 0, `'b'` = 1, ..., `'h'` = 7 (`charCodeAt(0) - 97`)

- Row: '1' = rank 1 → row index 7; '8' = rank 8 → row index 0

Center squares: columns 2-5 (c-f), rows 2-5 (3-6) = **c3-f6** = center 16 squares.

Analogy: Knight ko **chowk** pe khada karo. Corner mein bekaar hai. 🐎

🎯 Rook 7th Rank:

typescript

```
if ((move.color === 'w' && toRow === 7) || (move.color === 'b' && toRow === 2)) {  
  return 'Infiltrated Rook onto the powerful 7th rank!';  
}
```

7th rank = opponent ke pawns ki line. Rook wahan **bahut powerful** hoti hai.

Analogy: Tumne **enemy base** mein ghus ke Rook rakha. Destroy karega. 🔪

🌟 PART 8 — extractPV() — Anticipated Line

typescript

```
export function extractPV(game: Chess, maxPlies = 3): string[] {  
  const pv: string[] = [];  
  const tempGame = new Chess(game.fen());  
  
  for (let ply = 0; ply < maxPlies; ply++) {  
    const key = getPositionKey(tempGame.fen());  
    const entry = transpositionTable.get(key);  
    if (!entry || !entry.bestMove) break;  
  
    const move = entry.bestMove;  
    try {  
      const res = tempGame.move(move);  
      if (!res) break;  
      pv.push(res.san);  
    } catch {  
      break;  
    }  
  }  
}
```

```
return pv;
}
```

Concept: Principal Variation

Har TT entry mein **best move** hota hai. Chain karo:

text

```
Position A → TT.bestMove = "Nc6" → Position B
Position B → TT.bestMove = "Nc3" → Position C
Position C → TT.bestMove = "Nf6" → Position D
```

Result: ["Nc6", "Nc3", "Nf6"] = AI ka **anticipated line**.

Analogy: Chess commentary 🗣️: "AI ne Nc6 chala. Ab dekhte hain White kya karega... White Nc3 khel sakta hai, phir Black Nf6..."

Safety Guards:

1. `if (!entry || !entry.bestMove) break;` — TT mein chain khatam
2. `try/catch` — agar move invalid hua toh crash na ho
3. `if (!res) break;` — `move()` null return kare toh stop
4. `maxPlies` — hard limit (default 3)

Analogy: Tum **map** follow kar rahe ho. Agar road khatam, exit mila ya dead end — **ruko**. Aage mat jao. 🛑

`res.san` Kyu, `move.san` Kyu Nahi?

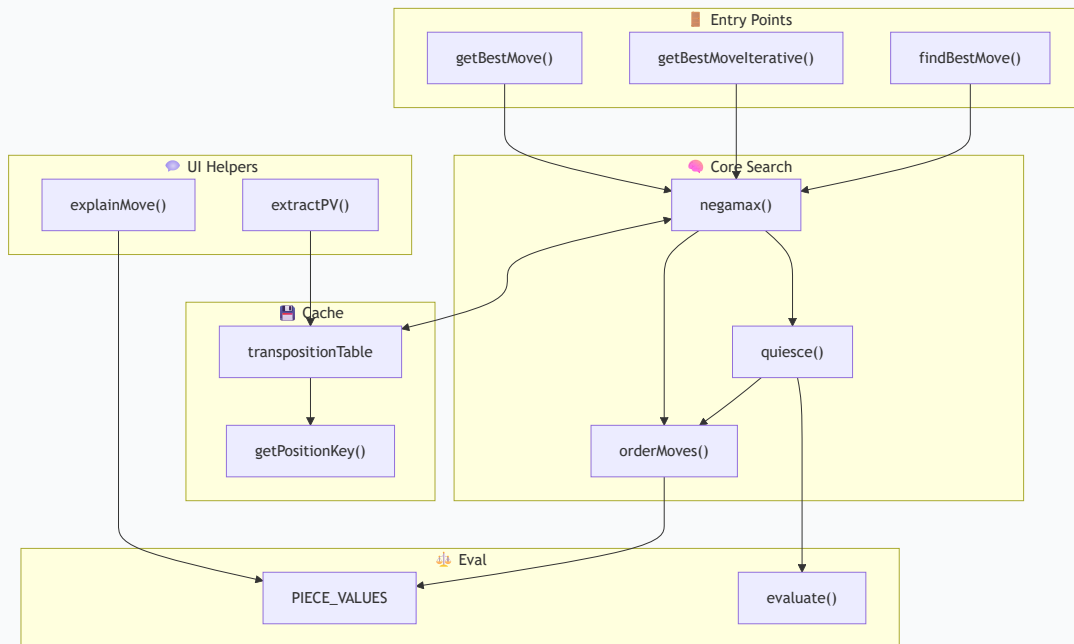
`res` = **applied move** (after chess.js validates). `res.san` = **proper SAN string** jaise `"Nc6"`, `"exd5"`, `"0-0"`.

Analogy: Tumne **raw input** diya tha, but **official receipt** mili. Wahi display karo. 🧾

🎯 Full Picture — Saare Functions Ka Relationship

Diagram

Code



🐛 Gotchas Is File Mein

⚠️ Gotcha #1: `game.move(move)` Accepts Move Object?

typescript

```
game.move(move); // Move object pass kiya
```

`chess.js` move object accept karta hai (jaise `{from: 'e2', to: 'e4'}` ya Move instance). But **flexible** raho — agar version change ho, toh check karo.

⚠️ Gotcha #2: `onNode` Double Call

typescript

```
if (onNode && onNode()) return 0; // ← Call #1
```

```
for (const move of moves) {
```

```

game.move(move);
const score = -negamax(game, depth - 1, -beta, -alpha, onNode);
game.undo();

if (onNode && onNode()) return 0; // ← Call #2 (in parent)
}

```

`onNode` **do jagah** call hota hai:

1. Function ke shuru mein (recursion se pehle)
2. Loop ke andar (har move ke baad)

Isse **node count double** ho sakta hai. Minor inefficiency, but harmless.

⚠ **Gotcha #3: `iterationBeta = Infinity` Hamesha**

```

typescript

const iterationBeta = Infinity; // Har depth pe same

```

Root pe **always full window** search karte hain ($\beta = \infty$). Advanced engines **aspiration window** use karte hain (chhota window, phir expand) — ye version use nahi karta.

⚠ **Gotcha #4: `checkTime()` Twice Per Move in Iterative**

```

typescript

for (const move of currentMoves) {
  if (checkTime()) { ... } // ← Before negamax
  // ...
  if (isTimeUp) { ... } // ← After negamax
}

```

`isTimeUp` check **negamax** ke andar `checkTime` set karta hai. Ye **double check** hai — safe but thoda redundant.

Ye file engine ka core hai 🧠 — `orderMoves` moves sort karta hai (TT + MVV), `negamax` recursive alpha-beta search karta hai TT cache ke saath, `quiesce` captures ko shaant karta hai, `getBestMoveIterative` time-budgeted deepening chalata hai, aur `explainMove` / `extractPV` user ko human-readable reasoning + anticipated line dete hain.

💡 Improvement Ideas

Idea	Kahan Change Karo
Aspiration windows	<code>getBestMoveIterative</code> mein narrow alpha-beta
Killer moves heuristic	<code>orderMoves</code> mein quiet moves ko bhi track
History heuristic	Global <code>Map<Move, score></code> maintain
Null move pruning	"Pass" move try karo, agar still good toh prune
Late move reduction	End moves shallow search karo
Better MVV-LVA	Attacker value bhi include karo
PV table separate	TT ke saath PV line explicitly store karo
Opening book	Known openings ke liye instant move

Bhai, ab 4 files ho gayi:

1. ✅ `ChessBoard.tsx` — UI + handlers
2. ✅ `chessAI.worker.ts` — Worker bridge
3. ✅ `chessAI.ts` — Engine core (ye file)
4. ⌚ `evaluation.ts` — Scoring
5. ⌚ `transposition.ts` — Cache

Agla kaunsa? Main **evaluation.ts** recommend karta hoon — kyunki abhi tak humne dekha *"score kaise use hota hai"*, ab dekhte hain *"score banta kaise hai"* (material + PST). 🎯

```
import { Move } from 'chess.js';

export type TTFlag = 'exact' | 'lower' | 'upper';

export interface TTEntry {
  depth: number;
  score: number;
  flag: TTFlag;
```



🧠 Thought for 1 second ▾

- User wants same detailed Hinglish explanation for transposition.ts file. Let me give a comprehensive breakdown with analogies, mermaid diagrams, and examples.

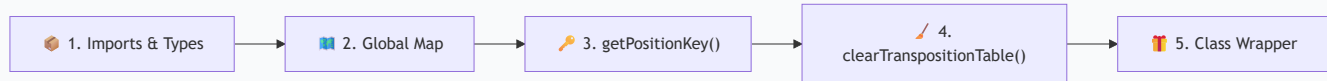
transposition.ts — Engine Ki Memory (Beginner Hinglish Guide)

Bhai, ye file **sabse chhoti** hai but **sabse critical** hai. Ye **engine ki yaaddasht** hai — jo positions pehle dekh chuki hai, unhe yaad rakhti hai taaki **dobara calculate na karna pade**.

Chalo isko **5 hisson** mein todte hain:

Diagram

Code



😬 Pehle: Transposition Table Aakhir Kya Hai?

🎯 Problem:

Chess mein **same position** tak **alag-alag move orders** se pahunch sakte ho. Example:

text

Path 1: 1. d4 Nf6 2. c4

Path 2: 1. c4 Nf6 2. d4

↑ Same position!

Bina cache ke, engine **same position** ko 2 baar search karega — **waste of time!**

🎯 Solution: Transposition Table

Har position ko **FEN string** se identify karo, aur uska **result** (score, best move, depth) memory mein store karo.

Analogy: Tumhare paas **maths ke difficult sawaal ka solution rough copy** mein likha hai. Exam mein same sawaal aaya toh **pura solve karne ki zaroorat nahi** — copy se

answer likh do. Time bachao! 📝⌚

🎯 Kyu "Transposition" Naam?

Transposition = "position ko badalna / re-arrange karna". Chess mein moves ka **order badalne** pe bhi same position aa sakti hai. Isliye "Transposition Table".

📦 PART 1 — Imports & Type Definitions

```
typescript

import { Move } from 'chess.js';

export type TTFlag = 'exact' | 'lower' | 'upper';

export interface TTEntry {
  depth: number;
  score: number;
  flag: TTFlag;
  bestMove?: Move | null;
}
```

🎯 TTFlag — 3 Kism Ke Answers

Ye **Union Type** hai — sirf 3 values allowed hain:

Flag	Matlab	Kab Set Hota Hai
'exact'	Pura accurate score mila	Normal case — pura search hua
'lower'	Score kam se kam itna toh hai (β cutoff)	Beta cutoff hua tha
'upper'	Score zyada se zyada itna hai (α fail-low)	Search incomplete thi

Analogy — Package Ka Weight:

- **Exact** = "Package **exactly** 5kg hai" ✅ (perfect info)
- **Lower** = "Package 5kg se zyada hai" ⬆️ (minimum known)
- **Upper** = "Package 5kg se kam hai" ⬇️ (maximum known)

🎯 TTEEntry Interface — Ek Position Ka "Memory Card"

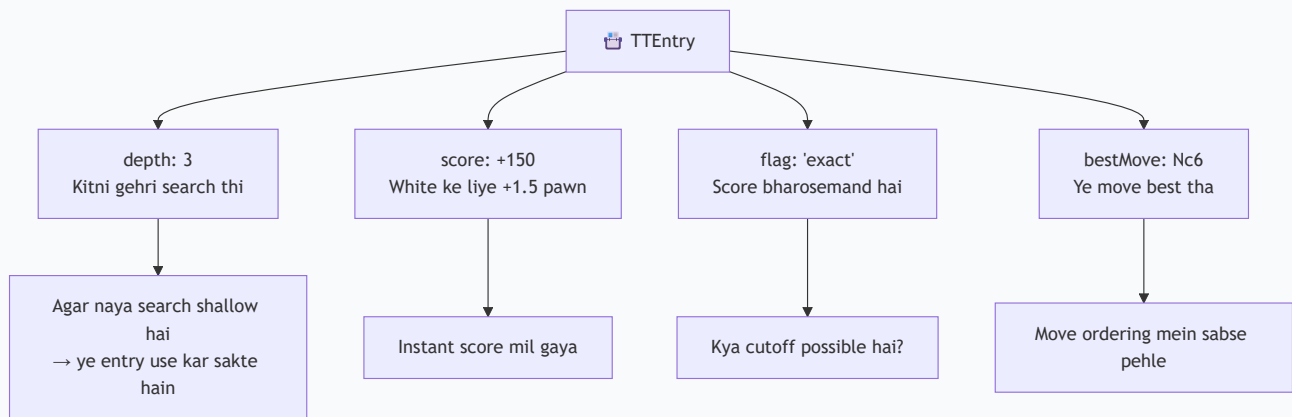
typescript

```
export interface TTEEntry {
  depth: number;           // Kitni depth tak search kiya
  score: number;           // Kya score mila
  flag: TTFlag;            // Score kitna reliable hai
  bestMove?: Move | null; // Best move kya tha
}
```

Har field ka kaam:

Diagram

Code



🎯 bestMove? Optional Kyu?

Kabhi kabhi **game over** position hoti hai (checkmate/draw), toh `bestMove` **null** ho sakta hai. Optional chaining (`?.`) se safe access.

Analogy: Har **visiting card** mein phone number nahi hota. Kabhi sirf email hota hai. 📄

📖 PART 2 — Global Map (Poori Duniya Ka Cache)

typescript

```
export const transpositionTable = new Map<string, TTEntry>();
```

🎯 Ye Ek Line Kya Karti Hai?

Ek **global Map** banati hai jo:

- **Key** = FEN string (position)
- **Value** = `TTEntry` (score, depth, best move)

Poori project isi ek Map ko share karta hai! `chessAI.ts` mein import hoke use hota hai.

🎯 Map vs Object — Kyu Map ?

Feature	Object	Map ✓
Key type	Sirf string/symbol	Kuch bhi
Size	Manual count	<code>.size</code> property
Performance (frequent add/delete)	Slow	Fast
Iteration order	Insertion order	Insertion order
Prototype pollution	Possible	Safe

Chess engine mein:

- **Hazaaron entries** add hote hain search ke dauran
- **Fast lookup** critical hai
- `Map` perfect fit! 🎯

Analogy: `Object` = **purani almari** (kuch cheezein atak jaati hain). `Map` = **naya smart cabinet** (kuch bhi rakh do, fast nikalta hai). 🗄️

🎯 Global Kyu? (Module Level)

typescript

```
export const transpositionTable = new Map<string, TTEntry>();
```

Ye **module ke top level** pe hai — matlab jab bhi ye file import hogi, **same instance** milega (module caching).

Fayda: `chessAI.ts`, `transposition.ts` — sab **same Map** share karte hain.

Nuksan: Testing mein **manual clear** karna padta hai warna **state leak** hota hai tests ke beech.

Analogy: Ye **shared whiteboard** hai classroom mein. Sab teachers (modules) likh sakte hain. But **naya class** (test) shuru karne se pehle **board saaf** karna zaroori hai. 🧼

🎯 Memory Concern?

Concern: Search karte-karte Map **bahut bada** ho sakta hai (lakhs of entries).

Reality: Modern JavaScript engines mein `Map` **millions of entries** easily handle kar sakta hai. But **memory leak** possible hai agar clear na karo.

Improvement idea: LRU (Least Recently Used) cache use karo taaki purani entries auto-delete ho jaayein.

🔑 PART 3 — `getPositionKey()` — FEN Ka "Aadhaar Card"

typescript

```
export function getPositionKey(fen: string): string {  
  const parts = fen.split(' ');
```

```
return parts.slice(0, 4).join(' ');  
}
```

🎯 Ye Function Kya Karta Hai?

FEN string leta hai aur pehle 4 parts nikaal ke wapas join karta hai.

🎯 FEN Ke 6 Parts Recap:

text

```
rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq e3 0 1  
                ↑ ↑ ↑  
                1 2 3
```

#	Part	Example	Importance
1	Board pieces	rnbqkbnr/pppp...	✅ Zaroori
2	Active color	b	✅ Zaroori
3	Castling rights	KQkq	✅ Zaroori
4	En-passant square	e3	✅ Zaroori
5	Halfmove clock	0	❌ Skip
6	Fullmove number	1	❌ Skip

🎯 Kyu Sirf Pehle 4 Parts?

Parts 5, 6 = "clocks" — ye position identity ka hissa nahi hain!

Example — 2 Positions:

text

```
A: rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq e3 0 1  
B: rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq e3 5 3
```

↑ ↑
clocks

Chess ke rules ke hisaab se, A aur B ek hi position hain. Sirf clocks alag hain (move number aur 50-move rule count).

Agar clocks **include** karo key mein, toh **same position match nahi hogi** = **cache MISS** = **waste!**

Analogy: Tum **Aadhaar card** se identity check karte ho. **Address change** ho gaya, kya tum ab doosre insaan ho? Nahi! Clocks = address. Position = tum ho. 🖨️

🎯 `split(' ')` — FEN Ko Pieces Mein Todna

typescript

```
const parts = fen.split(' ');  
// "rnbq... b KQkq e3 0 1".split(' ')  
// → ["rnbq...", "b", "KQkq", "e3", "0", "1"]
```

`split(' ')` string ko **space** pe todta hai → array.

Analogy: Ek **poora sentence** ko **words** mein todna. 📄 → 📄 📄 📄

🎯 `slice(0, 4)` — Pehle 4 Nikalo

typescript

```
parts.slice(0, 4)  
// ["rnbq...", "b", "KQkq", "e3"] ← Parts 5, 6 gaye
```

`slice(start, end)` — `start` inclusive, `end` exclusive.

🎯 `join(' ')` — Wapas Jodo

typescript

```
.join(' ')
```

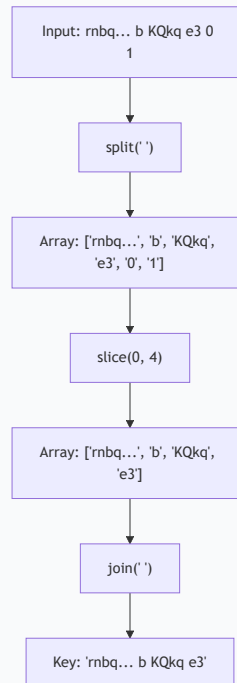
```
// "rnbq... b KQkq e3"
```

Result: Normalized 4-part FEN — ye hi cache key ban jaati hai.

Flowchart:

Diagram

Code



Real Example:

text

Input FEN: "rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq e3 0 1"

Output Key: "rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq e3"

Notice: 0 1 (last 2 parts) hat gaye! ✂️

Important Caveat: En-Passant Square

Kuch engines **en-passant square** ko bhi normalize karte hain — agar **koi pawn actually capture kar sakta hai** toh hi include karte hain.

Kyu? Kyunki FEN standard ke hisaab se, en-passant square **hamesha set hoti hai** jab pawn 2 square chali ho — chahe koi capture kar sake ya na kar sake.

Example:

text

Position: White just played e2-e4
FEN: ... b KQkq e3 ... ← e3 set hai

But agar Black ke paas koi pawn nahi hai jo d5 se capture kar sake,
toh e3 square irrelevant hai.

Is version mein ye simplification nahi hai — en-passant square **always included**. Chhota inefficiency (rare duplicate positions), but harmless.

Improvement idea: Advanced engines mein ye normalize karte hain.



PART 4 — clearTranspositionTable()

typescript

```
export function clearTranspositionTable(): void {  
  transpositionTable.clear();  
}
```



Ye Kya Karta Hai?

Poora Map khali kar deta hai. Sab cached entries **delete**.



Kab Call Karna Chahiye?

Situation	Kyu?
Naya game shuru	Purani positions ab relevant nahi
Unit tests ke beech	Test isolation (ek test doosre ko affect na kare)

Situation	Kyu?
Memory pressure	Agar Map bahut bada ho gaya
Debugging	Fresh state se start karna hai

🎯 Kyu Wrapper Function? (Direct `.clear()` Kyu Nahi?)

Direct `transpositionTable.clear()` bhi kaam karta, but:

1. **Naming clarity** — `clearTranspositionTable()` self-documenting
2. **Future-proof** — kabhi extra logic add karni ho (logging, metrics)
3. **Consistent API** — class ke `clear()` ke saath match

Analogy: Tum "bijli band karo" kehte ho, na ke "MCB switch flip karo". Same kaam, better naam. 💡

⚠️ Gotcha: Clear Sab Kuch Gayab Kar Deta Hai

Ye **selective delete** nahi karta. **Poora cache gone!**

Agar tumne 1000 positions cache ki thi aur ek clear call kiya — **sab gayab**. Next search **scratch se shuru** hogi.

Better approach for production:

```
typescript

// Selective eviction - sirf purani/shallow entries hatao
function pruneShallowEntries(minDepth: number): void {
  for (const [key, entry] of transpositionTable) {
    if (entry.depth < minDepth) transpositionTable.delete(key);
  }
}
```

🎁 PART 5 — TranspositionTable Class (Wrapper)

typescript

```
export class TranspositionTable {  
  private table = transpositionTable;  
  
  public get(key: string): TTEEntry | undefined {  
    return this.table.get(getPositionKey(key));  
  }  
  
  public set(key: string, entry: TTEEntry): void {  
    this.table.set(getPositionKey(key), entry);  
  }  
  
  public clear(): void {  
    this.table.clear();  
  }  
  
  public size(): number {  
    return this.table.size;  
  }  
}
```

🎯 Class Kyu? Functions Toh Already Hain!

Do reasons:

1. **Interface Conventions** — Project design mein class-based API better lagti hai
2. **Testability** — Class instances **mock** karna easy hota hai
3. **Future extensibility** — Kal ko instance-level state add kar sakte hain

🎯 `private table = transpositionTable;`

Ye line **class ke andar** global Map ka **reference** store karti hai. **Naya Map nahi banati** — same instance!

Important: `private` matlab class ke bahar se direct access nahi. `get`, `set`, `clear` methods se hi.

Analogy: Tumhare ghar mein **almari** hai. `private table` = almari **andar** hai. Bahar wale ko **darwaza** (method) se hi access milega. 🚪

`get()` — Auto Normalization

typescript

```
public get(key: string): TTEntry | undefined {  
    return this.table.get(getPositionKey(key));  
}
```

Jadoo: Caller **poori FEN** bhej sakta hai, ye **auto-normalize** karke Map mein lookup karega!

typescript

```
const tt = new TranspositionTable();  
tt.get("rnbq... b KQkq e3 0 1"); //  Full FEN do  
// Internally: getPositionKey() → "rnbq... b KQkq e3" → Map Lookup
```

Fayda: Caller ko yaad nahi rakhna ki pehle normalize karo. **User-friendly API!**

Analogy: Restaurant mein tum "ek coffee" bolte ho. Waiter khud samajh jaata hai ki size, sugar, milk kaunsa. ☕

`set()` — Auto Normalization (Phir Se)

typescript

```
public set(key: string, entry: TTEntry): void {  
    this.table.set(getPositionKey(key), entry);  
}
```

Same logic — store karte waqt bhi normalize karta hai.

`clear()` — Delegation

typescript

```
public clear(): void {  
    this.table.clear();  
}
```

Simple delegation — class ka method **global Map ke clear** ko call karta hai.

size() — Map Kitni Entries Hain?

typescript

```
public size(): number {  
    return this.table.size;  
}
```

Ye Useful Hai:

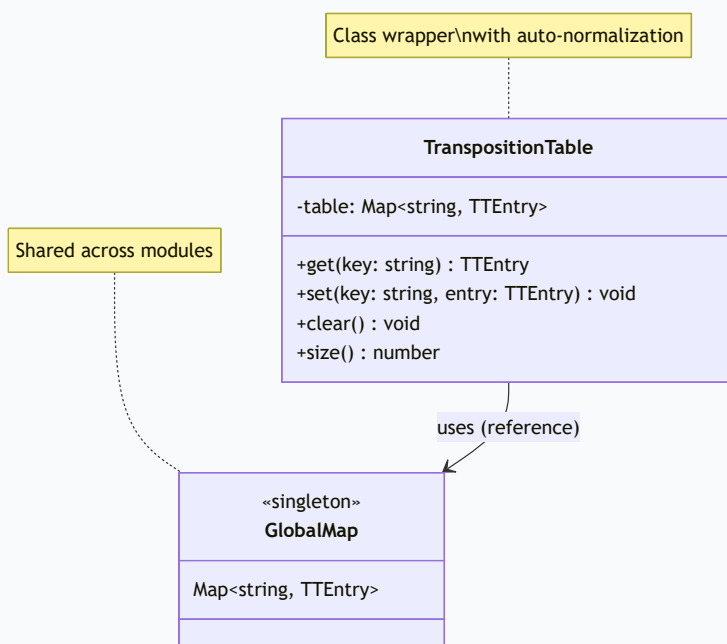
- **Debugging:** "Kitni positions cache hui?"
- **UI telemetry:** "TT size: 15,420 entries" dikha sakte ho
- **Testing:** "Expected size match karta hai?"

Analogy: Tumhari **diary mein kitne pages likhe hain**, count karna. 📖

Class Ka Diagram:

Diagram

Code



Do APIs — Kab Kaunsa Use Karo?

API Style

Kab Use Karo

Functions (`getPositionKey` ,
`transpositionTable`)

Direct control chahiye, `chessAI.ts` jaise
internal code

Class (`new TranspositionTable()`)

Clean interface chahiye, external
consumers, testing

Dono same Map pe kaam karte hain! Sirf API different hai.

Analogy: Ek hi car — **manual gear** bhi hai, **automatic** bhi. Dono se drive kar sakte ho,
bas preference ka farq. 🚗

⚠️ Gotcha: Class Ka `private table` Actually Shared Hai

typescript

```
private table = transpositionTable;
```

Agar tum **2 instances** banao:

typescript

```
const tt1 = new TranspositionTable();  
const tt2 = new TranspositionTable();  
tt1.set("position A", ...);  
console.log(tt2.size()); // 🧐 1 — kyunki same Map share kar rahe hain!
```

Kyu? Dono `tt1` aur `tt2` same global Map pe point kar rahe hain.

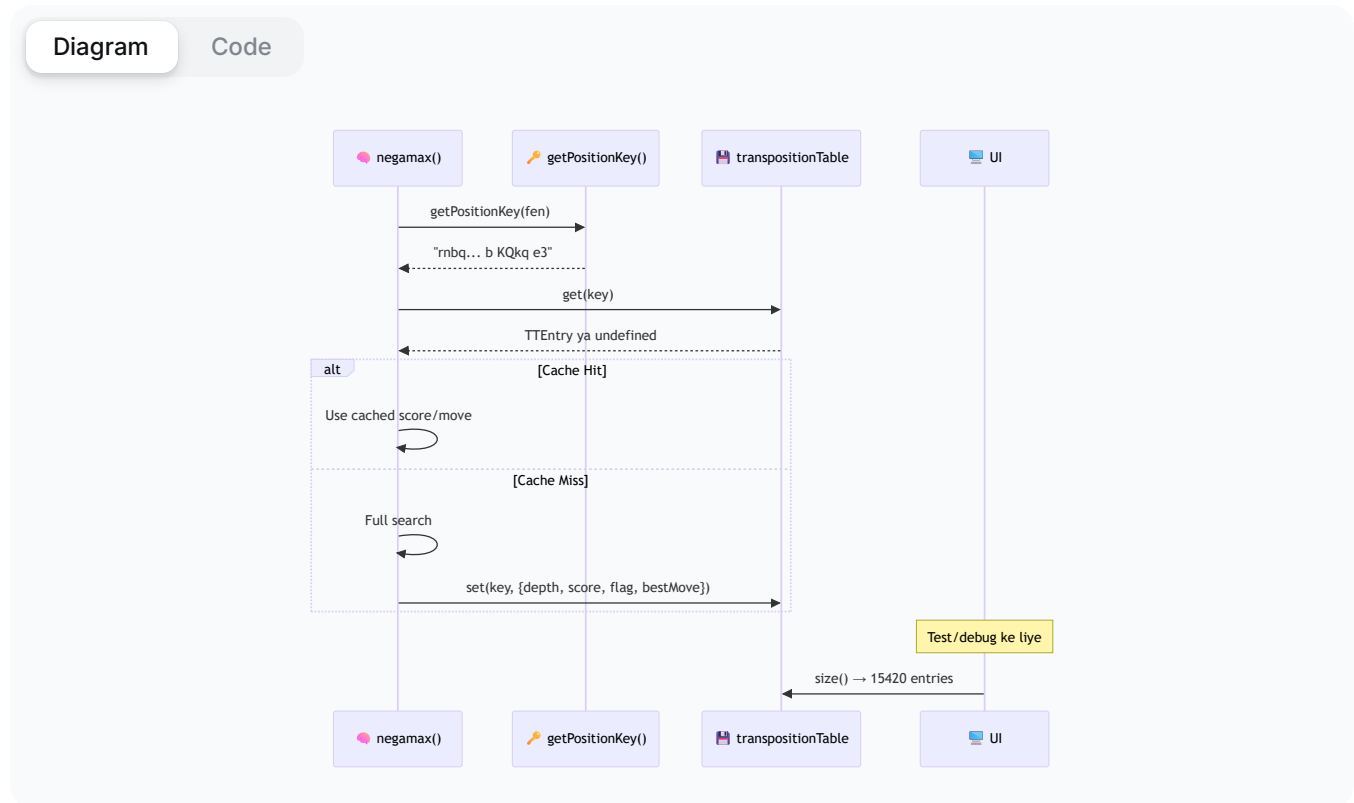
Agar separate instances chahiye (testing ke liye):

typescript

```
export class TranspositionTable {  
  private table = new Map<string, TTEntry>(); // ← Naya Map!  
  // ...  
}
```

But ye project mein **intentional** hai — kyunki hum **cache share** karna chahte hain across engine calls.

🎯 Full Picture — TT Kaise Use Hota Hai



🐛 Gotchas Is File Mein

⚠️ Gotcha #1: Global State Testing Mein Leak

typescript

```
// Test 1:
findBestMove(fen1, 3); // TT mein entries daal di

// Test 2:
findBestMove(fen2, 3); // Purani entries se affect ho sakta hai!
```

```
// Fix:
beforeEach(() => clearTranspositionTable());
```

⚠️ Gotcha #2: En-Passant Square Include Hamesha

typescript

```
// FEN: "... b KQkq e3 0 1"
// Key: "... b KQkq e3" ← e3 include
```

Agar **koi** en-passant capture nahi kar sakta, toh bhi **include** hoti hai. **Chhoti inefficiency** — different keys for same logical position.

⚠️ Gotcha #3: `size()` Memory Nahi Batata

typescript

```
tt.size(); // 15420 entries
```

Ye sirf **count** hai. **Memory usage** nahi. Har entry ka **actual size** ~200 bytes hota hai. So 15420 entries $\approx$ 3 MB. But JS internals vary karte hain.

⚠️ Gotcha #4: No LRU Eviction

Purani entries **kabhi auto-delete nahi** hoti. Long-running session mein Map **grow** karta rehta hai.

Fix:

typescript

```
if (transpositionTable.size > 100000) {
  clearTranspositionTable(); // Ya LRU evict karo
}
```

Ye file engine ki yaaddasht hai 📁 — FEN ko normalize karke (clocks hata ke) unique key banati hai, us key pe search results store karti hai (score, depth, flag, best move), aur ek global Map ke through poore project ko share karti hai — jisse identical positions dobara search na karni pade aur Alpha-Beta cutoffs 2x tez ho jaayein.

💡 Improvement Ideas

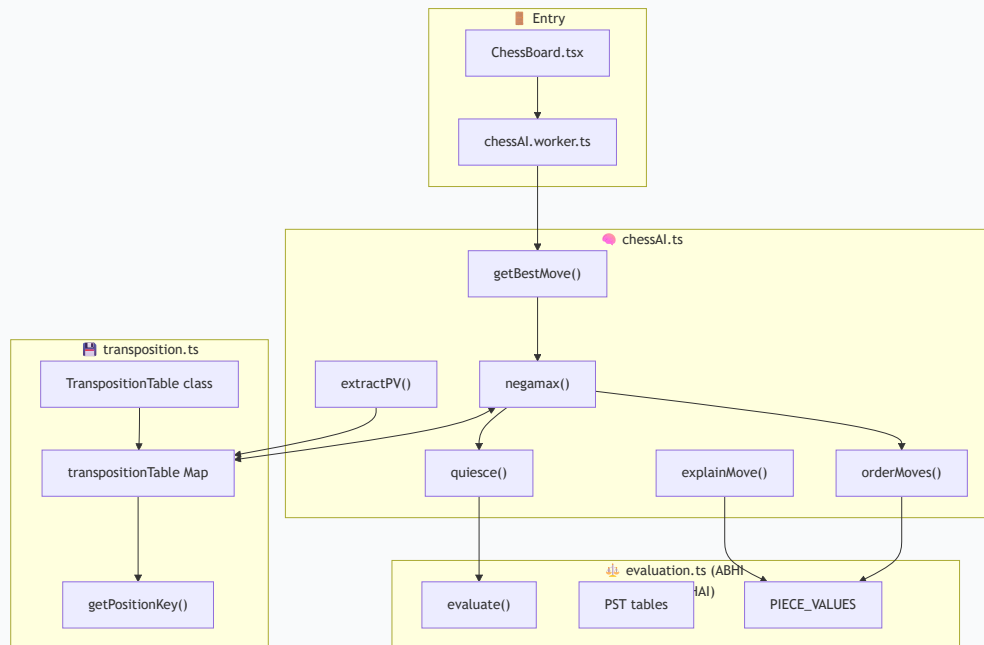
Idea	Kya Add Karo
LRU Cache	Oldest entries auto-evict karo
TT Size limit	Max entries set karo, purge karo
En-passant normalization	Sirf relevant e.p. include karo
Zobrist Hashing	FEN string ki jagah integer hash (faster)
Two-tier TT	Always-replace + depth-preferred buckets
TT hit rate telemetry	UI pe "TT hits: 45%" dikhao
Persistent across sessions	IndexedDB mein save karo
Separate TT for quiescence	Capture search ke liye alag cache



Ab Tak Ka Complete Picture

Diagram

Code



Bhai, ab tak 5 files ho gayi:

1. `ChessBoard.tsx` — UI + handlers
2. `chessAI.worker.ts` — Worker bridge
3. `chessAI.ts` — Engine core
4. `transposition.ts` — Cache (ye file)
5. `evaluation.ts` — **BAKHRA HAI!** 🎯

Agla file `evaluation.ts` chahiye? Wahan **material values + PST tables + endgame detection** sab dekhenge — **sabse mazedaar** part kyunki wahi engine ko "**samajh**" deta hai ki position achi hai ya buri! 💬 ✨



import { Chess, PieceSy...
TXT 7.37KB

same for this also

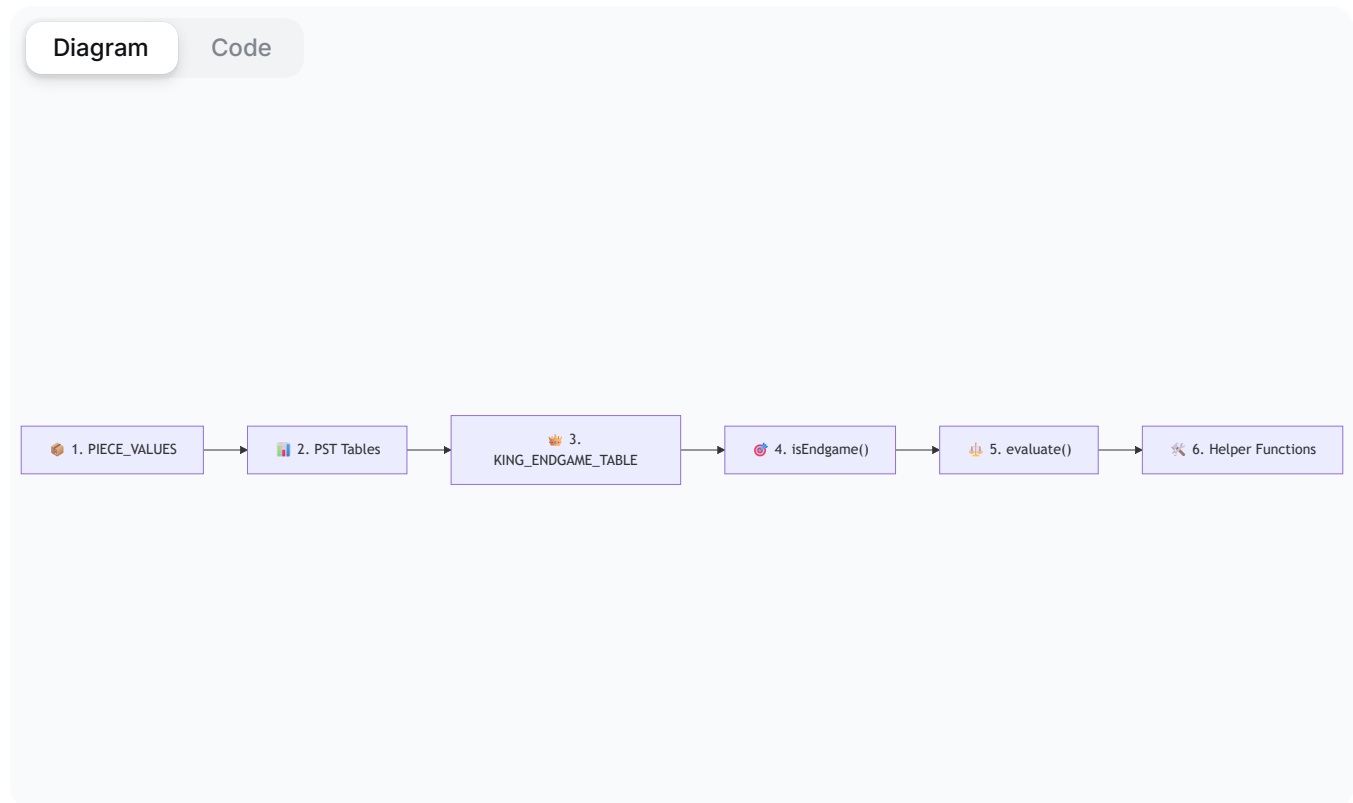
Thought for 1 second >

evaluation.ts — Engine Ki Aankhein (Beginner Hinglish Guide)

Bhai, ye file **sabse mazedaar** hai! 🌈 Kyunki ab tak humne dekha ki **score kaise use hota hai** (search mein), ab dekhte hain **score banta kaise hai**.

Ye file engine ko "**samajh**" deti hai — ki position **achi hai ya buri**.

Chalo isko **6 hisson** mein todte hain:



😬 Pehle: Evaluation Aakhir Kya Hai?

🎯 Problem:

Engine search karta hai hazaaron positions. But **kis position ko choose kare?** Uske liye har position ko ek **number** chahiye — "*ye position kitni achi hai*".

Ye number = **Evaluation Score**.

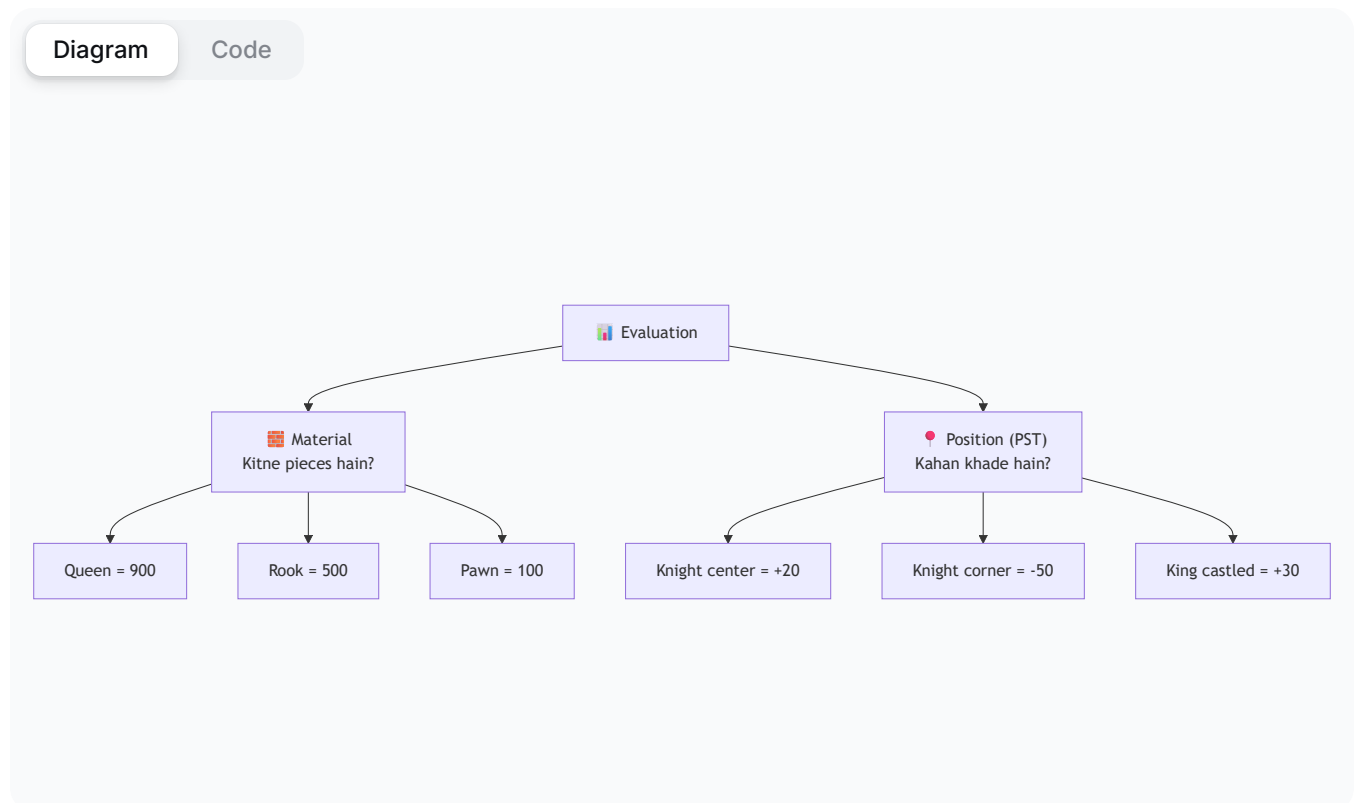
🎯 Score Ka Format:

Centipawns mein score hota hai (1 pawn = 100 centipawns).

Score	Matlab
+100	White 1 pawn se aage hai
+900	White 1 Queen se aage hai
-50	White aadha pawn peeche hai
0	Equal position
-Infinity	Checkmate (koi jeet gaya)

Analogy: Ye jaise **cricket ka scoreboard** 🏏 . Har ball ke baad score update hota hai.
 Chess mein har position ka apna "score" hota hai.

🎯 Evaluation Ke 2 Components:



Analogy: Ek cricket team:

- **Material** = kitne **bade players** hain (Virat, Rohit)
- **Position** = **kahan khade** hain (cover pe ya slip pe)

Dono milke team ki **strength** banate hain!



PART 1 — PIECE_VALUES (Material Values)

typescript

```
export const PIECE_VALUES: Record<PieceSymbol, number> = {
  p: 100,    // Pawn
  n: 320,    // Knight
  b: 330,    // Bishop
  r: 500,    // Rook
  q: 900,    // Queen
  k: 20000,  // King
};
```



Ye Numbers Kahan Se Aaye?

Ye **standard chess values** hain jo centuries se use ho rahi hain:

Piece	Value	Kyu?
Pawn	100	Basic unit
Knight	320	3 pawns ke barabar
Bishop	330	Knight se thoda better (open positions mein)
Rook	500	5 pawns
Queen	900	9 pawns
King	20000	Game over piece — value infinite



Knight (320) vs Bishop (330) — Ye 10 Ka Farq Kyu?

Dono "minor pieces" hain — 3 pawns ke barabar. But **bishop thoda better** maana jaata hai:

- **Bishop** = Long range, open diagonals pe powerful
- **Knight** = Short range, but center mein powerful

10 centipawn difference = 0.1 pawn — bahut chhota tie-breaker.

Analogy: Do dost hain — ek **cricket** mein acha, ek **football** mein. Overall almost same, but situation ke hisaab se farq. 🏏️ ⚽️

🎯 King = 20000 Kyu? (Infinite Nahi?)

Kyu nahi infinite? Kyunki:

1. Agar king **infinite** ho, toh $+20000 - 20000 = 0$ — draw ho jayega!
2. **20000** = "effectively infinite" but **calculable**
3. Dono kings hamesha board pe hote hain (except checkmate), toh **usually cancel** ho jaate hain

Checkmate detection alag se hota hai (`evaluate` mein) — `isCheckmate()` check karta hai.

Analogy: King ko "priceless" keh sakte hain, but account mein **20000 rupees** likh do. Dono kings ka same hai, toh **net 0** — baaki pieces ka farq decide karega. 🏰

🎯 `Record<PieceSymbol, number>` Type

TypeScript magic! Ye kehta hai:

- **Key** = `PieceSymbol` (`'p' | 'n' | 'b' | 'r' | 'q' | 'k'`)
- **Value** = `number`

Agar tum `PIECE_VALUES.x` likho, **TypeScript error** dega — kyunki `x` valid piece nahi hai.

Analogy: Ek **dictionary** jisme sirf 6 allowed words hain. Koi naya word add nahi kar sakte. 📖

PART 2 — PST Tables (Piece-Square Tables)

PST Kya Hai?

Har piece ke liye 8x8 ka grid — batata hai ki **kis square pe kitna bonus/penalty** milega.

Format:

text

```
Table[row][col]
```

```
row 0 = Rank 8 (black ki back rank)
```

```
row 7 = Rank 1 (white ki back rank)
```

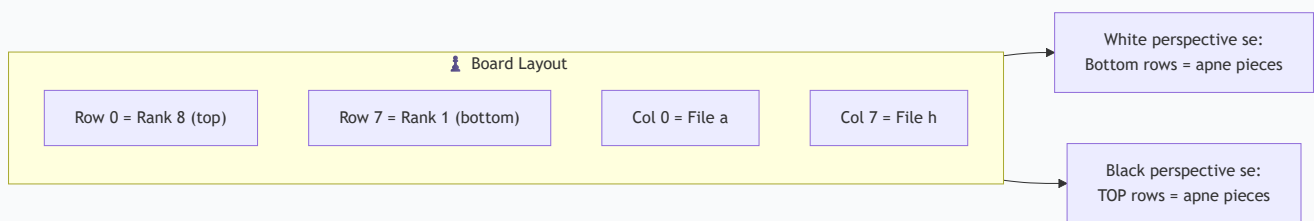
```
col 0 = File a
```

```
col 7 = File h
```

Coordinate System:

Diagram

Code



Confusion point: Ye tables **White ke perspective** se likhi gayi hain. Black ke liye **flip** karni padti hai (later `evaluate()` mein dekhenge).

Analogy: Tumne **apne ghar ka map** banaya (White). Agar dost (Black) ko dena hai, toh **ulta** karna padega. 🏠

 Pawn PST — Analyze Karte Hain:

typescript

```
p: [
  [ 0, 0, 0, 0, 0, 0, 0, 0], // Rank 8 (promotion)
  [ 50, 50, 50, 50, 50, 50, 50, 50], // Rank 7 (almost promoted!)
  [ 10, 10, 20, 30, 30, 20, 10, 10], // Rank 6
  [ 5, 5, 10, 25, 25, 10, 5, 5], // Rank 5
  [ 0, 0, 0, 20, 20, 0, 0, 0], // Rank 4 (center!)
  [ 5, -5, -10, 0, 0, -10, -5, 5], // Rank 3
  [ 5, 10, 10, -20, -20, 10, 10, 5], // Rank 2 (start)
  [ 0, 0, 0, 0, 0, 0, 0, 0], // Rank 1
]
```

Reading Kaise Kare:

Row	Value	Matlab
Row 0 (Rank 8)	0	Pawn yahan = promotion ho chuki, ab pawn nahi
Row 1 (Rank 7)	+50	Pawn almost promote — bahut acha!
Row 4 (Rank 4)	+20 center mein, 0 side pe	Center control bonus
Row 6 (Rank 2)	-20 center mein	Undeveloped center pawn = penalty
Row 7 (Rank 1)	0	Pawn yahan = promotion ho chuki

 Key Insights:

- 1. **Center pawns (d4, e4)** = +20 bonus — center control

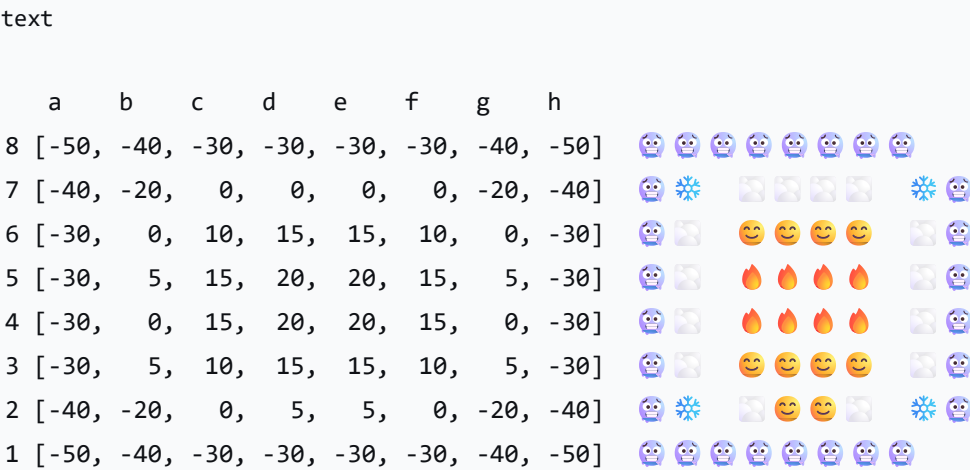
2. 7th rank pawn = +50 — promotion ke kareeb
3. d2, e2 pawns = -20 — undeveloped (blocked)
4. Edge pawns (a, h) = 0 — neutral

Analogy: Soldier ko **front line** bhejo (7th rank) = hero. **Ghar pe** rakho (2nd rank) = bekaar. **Center pe** khada hai = area control. 🏆

🎯 Knight PST — Analyze Karte Hain:

```
typescript
n: [
  [-50, -40, -30, -30, -30, -30, -40, -50], // Rank 8 - corner worst
  [-40, -20, 0, 0, 0, 0, -20, -40], // Rank 7
  [-30, 0, 10, 15, 15, 10, 0, -30], // Rank 6
  [-30, 5, 15, 20, 20, 15, 5, -30], // Rank 5 ← BEST
  [-30, 0, 15, 20, 20, 15, 0, -30], // Rank 4 ← BEST
  [-30, 5, 10, 15, 15, 10, 5, -30], // Rank 3
  [-40, -20, 0, 5, 5, 0, -20, -40], // Rank 2
  [-50, -40, -30, -30, -30, -30, -40, -50], // Rank 1 - corner worst
]
```

🎯 Visual Heatmap:



Legend:

- 🔥 = +20 (best — d4, d5, e4, e5)
- 😊 = +15, +10 (good)
- ☁ = 0 (neutral)
- ❄ = -20, -40 (bad)
- 🤖 = -50 (worst — corners)

🎯 Famous Quote:

"A knight on the rim is dim." 🐎

Knight **corner/edge** pe = -50. Knight **center** pe = +20. Total difference: **70 centipawns** = 0.7 pawn!

🎯 Kyu Center Itna Important?

Knight **center** pe = 8 squares attack karta hai

Knight **corner** pe = sirf 2 squares attack karta hai

Difference: 4x mobility!

Analogy: Tum **chowk (center)** pe khade ho → 8 directions mein ja sakte ho. **Corner** mein → sirf 2 directions. Soldier ki tarah — **chowk pe** khada hai toh pura area dominate karega. 🎯

🎯 Bishop PST:

typescript

```
b: [
  [-20, -10, -10, -10, -10, -10, -10, -20],
  [-10,  0,  0,  0,  0,  0,  0, -10],
  [-10,  0,  5, 10, 10,  5,  0, -10],
  [-10,  5,  5, 10, 10,  5,  5, -10],
  [-10,  0, 10, 10, 10, 10,  0, -10],
  [-10, 10, 10, 10, 10, 10, 10, -10],
  [-10,  5,  0,  0,  0,  0,  5, -10],
```

```
[ -20, -10, -10, -10, -10, -10, -10, -20 ],  
]
```

Bishop ki story:

- **Edges/corners** = penalty (mobility kam)
- **Center diagonals** = bonus (long range attack)
- Difference **kam** hai (knight jitna dramatic nahi) — kyunki bishop **long range** hai, door se bhi attack kar sakta hai.

Rook PST:

typescript

```
r: [  
  [ 0, 0, 0, 0, 0, 0, 0, 0], // Rank 8  
  [ 5, 10, 10, 10, 10, 10, 10, 5], // Rank 7 ← 7th rank bonus!  
  [ -5, 0, 0, 0, 0, 0, 0, -5],  
  [ -5, 0, 0, 0, 0, 0, 0, -5],  
  [ -5, 0, 0, 0, 0, 0, 0, -5],  
  [ -5, 0, 0, 0, 0, 0, 0, -5],  
  [ -5, 0, 0, 0, 0, 0, 0, -5],  
  [ 0, 0, 0, 5, 5, 0, 0, 0], // Rank 1 (d1, e1 bonus)  
]
```

Insights:

1. Row 1 (Rank 7) = +10 bonus — 7th rank rook!
2. Row 0 (Rank 8) = 0 — enemy back rank (kabhi acha, kabhi bura)
3. Row 6 (Rank 2) = -5 — passive rook
4. Row 7 (Rank 1) = 0, but d1/e1 = +5 — connected rooks

7th Rank Kyu Powerful?

Opponent ke pawns ki line = 7th rank. Wahan rook:

- Pawns ko pin kar sakta hai

- King ko **trap** kar sakta hai
- Promotion **rok** sakta hai

Analogy: Tumne **enemy base** mein ghus ke Rook rakha. Destroy karega! ✂️

🎯 Queen PST:

typescript

```
q: [
  [-20, -10, -10, -5, -5, -10, -10, -20],
  [-10, 0, 0, 0, 0, 0, 0, -10],
  [-10, 0, 5, 5, 5, 5, 0, -10],
  [-5, 0, 5, 5, 5, 5, 0, -5],
  [ 0, 0, 5, 5, 5, 5, 0, -5],
  [-10, 5, 5, 5, 5, 5, 0, -10],
  [-10, 0, 5, 0, 0, 0, 0, -10],
  [-20, -10, -10, -5, -5, -10, -10, -20],
]
```

Queen ki story:

- **Center pe** thoda bonus (+5)
- **Corner pe** penalty (-20)
- **Difference chhota** hai — kyunki Queen **already powerful** hai, position se utna farq nahi padta.

Kyu chhote numbers? Queen ko **jaldi** mat active karo (opening mein). Isliye moderate values.

🎯 King PST (Middlegame):

typescript

```
k: [
  [-30, -40, -40, -50, -50, -40, -40, -30], // Rank 8 - exposed = BAD
```

```

[-30, -40, -40, -50, -50, -40, -40, -30], // Rank 7 – exposed
[-30, -40, -40, -50, -50, -40, -40, -30], // Rank 6
[-30, -40, -40, -50, -50, -40, -40, -30], // Rank 5
[-20, -30, -30, -40, -40, -30, -30, -20], // Rank 4
[-10, -20, -20, -20, -20, -20, -20, -10], // Rank 3
[ 20, 20, 0, 0, 0, 0, 20, 20], // Rank 2 ← CASTLED
[ 20, 30, 10, 0, 0, 10, 30, 20], // Rank 1 ← CASTLED
]

```

🎯 Ye Table Ulta Hai!

King ki special property: Opening mein king **corner mein safe** rehta hai (behind pawns).
Center mein **exposed**.

Positive values (top-right corner):

- Row 7, Col 1 (b1) = +30 ← **castled kingside/queenside!**
- Row 7, Col 2 (c1) = +10 ← Castled position
- Row 7, Col 6 (g1) = +30 ← **castled kingside!**
- Row 7, Col 7 (h1) = +20 ← Behind pawns

Negative values:

- Row 0-3 (Ranks 5-8) = -30 to -50 ← **exposed king**

🎯 Visual:

text

	a	b	c	d	e	f	g	h	
8	[-30,	-40,	-40,	-50,	-50,	-40,	-40,	-30]	💀💀💀💀💀💀💀💀
7	[-30,	-40,	-40,	-50,	-50,	-40,	-40,	-30]	💀💀💀💀💀💀💀💀
6	[-30,	-40,	-40,	-50,	-50,	-40,	-40,	-30]	💀💀💀💀💀💀💀💀
5	[-30,	-40,	-40,	-50,	-50,	-40,	-40,	-30]	💀💀💀💀💀💀💀💀
4	[-20,	-30,	-30,	-40,	-40,	-30,	-30,	-20]	😬😬😬😬😬😬😬😬
3	[-10,	-20,	-20,	-20,	-20,	-20,	-20,	-10]	😬😬😬😬😬😬😬😬
2	[20,	20,	0,	0,	0,	0,	20,	20]	🛡️🛡️🛡️🛡️🛡️🛡️🛡️🛡️
1	[20,	30,	10,	0,	0,	10,	30,	20]	🛡️🏰🏰🛡️🛡️🛡️🏰🏰🛡️

- 🏰 = Castle position (b1, c1, g1) — best!
- 🛡️ = Behind pawns — safe
- 💀 = Exposed in enemy territory — worst!

Analogy: King = Raja 🏰. Raja ko **fort ke andar** (castled) rakho. Jung ke maidan mein khula chhod do toh maar diya jayega!

👑 PART 3 — KING_ENDGAME_TABLE

typescript

```
export const KING_ENDGAME_TABLE: number[][] = [  
  [-50, -40, -30, -20, -20, -30, -40, -50], // Rank 8  
  [-30, -20, -10, 0, 0, -10, -20, -30], // Rank 7  
  [-30, -10, 20, 30, 30, 20, -10, -30], // Rank 6  
  [-30, -10, 30, 40, 40, 30, -10, -30], // Rank 5 ← CENTER  
  [-30, -10, 30, 40, 40, 30, -10, -30], // Rank 4 ← CENTER  
  [-30, -10, 20, 30, 30, 20, -10, -30], // Rank 3  
  [-30, -30, 0, 0, 0, 0, -30, -30], // Rank 2  
  [-50, -30, -30, -30, -30, -30, -30, -50], // Rank 1  
];
```

🎯 Ye Table Kyu Alag Hai?

Endgame mein king ka role badal jaata hai:

- **Opening/Middlegame:** King **safe** corner mein (castled)
- **Endgame:** King **active** center mein (attacking + supporting pawns)

Kyu? Kyunki endgame mein **kam pieces** hote hain — king ko attack ka **danger nahi**, toh wo **fighter** ban sakta hai!

Analogy:

- **Opening** = Jung ke maidan mein Raja **fort** mein rehta hai (safe)
- **Endgame** = Jung khatam, Raja **khud** maidan mein aata hai (pawns ko support karne)



🎯 Visual Heatmap:

text

	a	b	c	d	e	f	g	h	
8	[-50, -40, -30, -20, -20, -30, -40, -50]	🧊	🧊	❄️	🧊	🧊	❄️	🧊	🧊
7	[-30, -20, -10, 0, 0, -10, -20, -30]	🧊	❄️	🧊	🧊	🧊	❄️	🧊	🧊
6	[-30, -10, 20, 30, 30, 20, -10, -30]	🧊	🧊	😊	🔥	🔥	😊	🧊	🧊
5	[-30, -10, 30, 40, 40, 30, -10, -30]	🧊	🧊	🔥	💥	💥	🔥	🧊	🧊
4	[-30, -10, 30, 40, 40, 30, -10, -30]	🧊	🧊	🔥	💥	💥	🔥	🧊	🧊
3	[-30, -10, 20, 30, 30, 20, -10, -30]	🧊	🧊	😊	🔥	🔥	😊	🧊	🧊
2	[-30, -30, 0, 0, 0, 0, -30, -30]	🧊	🧊	🧊	🧊	🧊	🧊	🧊	🧊
1	[-50, -30, -30, -30, -30, -30, -30, -50]	🧊	🧊	🧊	🧊	🧊	🧊	🧊	🧊

🌟 = +40 (best — d4, d5, e4, e5) — King centralization!

🎯 Rule of Thumb:

Endgame mein King = ek extra piece! Use it actively.

🎯 PART 4 — isEndgame() (Endgame Detection)

typescript

```
export function isEndgame(game: Chess): boolean {
  let queens = 0;
  let minorPieces = 0;
  const board = game.board();

  for (let r = 0; r < 8; r++) {
    for (let c = 0; c < 8; c++) {
      const piece = board[r][c];
      if (!piece) continue;
      if (piece.type === 'q') queens++;
      if (piece.type === 'r' || piece.type === 'b' || piece.type === 'n') minorPieces++;
    }
  }
}
```

```
return queens === 0 || (queens <= 2 && minorPieces <= 2);  
}
```

🎯 Ye Function Kya Karta Hai?

Poora board scan karta hai, aur check karta hai:

1. Kitni queens hain?
2. Kitne minor pieces hain? (Rook, Bishop, Knight)

Endgame Conditions:

typescript

```
return queens === 0 || (queens <= 2 && minorPieces <= 2);
```

Condition

Matlab

queens === 0

Koi queen nahi — definitely endgame

queens <= 2 && minorPieces <= 2

2 ya kam queens aur 2 ya kam minor pieces
— endgame

🎯 Kyu Queens Pehle?

Queens sabse powerful pieces hain. Agar queens hain toh **attack possible** hai — king safe corner mein rehna chahiye. Queen trade hone ke baad **endgame** aata hai.

Analogy: Jungle mein **sher (queen)** hai toh Raja **ghar mein** rahe. Sher chala gaya, ab Raja **khulke ghoom** sakta hai. 🦁

🎯 (queens <= 2 && minorPieces <= 2) — Ye Kya?

2 ya kam queens = dono side ke paas 1-1 queen, ya koi queen nahi

2 ya kam minor pieces = bahut kam material — endgame!

Example:

- Q + R + R vs Q + R + R = **Not endgame** (bahut material)
- Q vs Q = **Not endgame** (queens still there)
- R + P vs R + P = **Endgame** ✅
- Q + N vs Q = **Not endgame** (queens present)
- Only Kings = **Endgame** ✅

🎯 Loop Optimization?

Ye code 64 squares scan karta hai har evaluation pe. Ye **thoda slow** hai.

Better: `game.board()` se pieces directly nikaalo, ya cached counts rakho.

But **64 iterations** = trivial cost vs search. **Acceptable.**

Improvement idea: Incremental material tracking — har move pe sirf change track karo.

⚖️ PART 5 — evaluate() (Main Function)

typescript

```
export function evaluate(game: Chess): number {
  // 🎯 Special cases
  if (game.isCheckmate()) return -Infinity;
  if (game.isDraw()) return 0;

  // 🎯 Setup
  let whiteScore = 0;
  let blackScore = 0;
  const board = game.board();
  const endgame = isEndgame(game);

  // 🎯 64 squares scan
  for (let r = 0; r < 8; r++) {
    for (let c = 0; c < 8; c++) {
      const piece = board[r][c];
      if (!piece) continue;
    }
  }
}
```

```

const materialVal = PIECE_VALUES[piece.type] ?? 0;
let pstVal = 0;

// 🎯 King endgame table ka special case
if (piece.type === 'k' && endgame) {
  pstVal = piece.color === 'w'
    ? KING_ENDGAME_TABLE[r][c]
    : KING_ENDGAME_TABLE[7 - r][c];
} else {
  const table = PST[piece.type];
  pstVal = piece.color === 'w'
    ? table[r][c]
    : table[7 - r][c];
}

const totalVal = materialVal + pstVal;
if (piece.color === 'w') whiteScore += totalVal;
else blackScore += totalVal;
}
}

// 🎯 Side-to-move perspective
const score = whiteScore - blackScore;
return game.turn() === 'w' ? score : -score;
}

```

🎯 Step-by-Step Breakdown:

Step 1: Special Cases

typescript

```

if (game.isCheckmate()) return -Infinity;
if (game.isDraw()) return 0;

```

Kyu?

- **Checkmate** = worst possible score for side to move (`-Infinity`)
- **Draw** = neutral (`0`)

Analogy: Match khatam hone pe score **freeze** ho jaata hai. Checkmate = haar, Draw = tie.

Step 2: Endgame Detection

typescript

```
const endgame = isEndgame(game);
```

Ek baar calculate karo — har piece pe dobara nahi.

Analogy: Ek baar **sabzi kaat lo**, baar-baar nahi. 🥒

Step 3: 64 Squares Scan

typescript

```
for (let r = 0; r < 8; r++) {  
  for (let c = 0; c < 8; c++) {  
    const piece = board[r][c];  
    if (!piece) continue; // ⚡ Empty square skip  
    // ...  
  }  
}
```

Kyu 8x8? Board `board()` se **2D array** milta hai — 8 rows, 8 columns.

`if (!piece) continue;` — **empty square skip** — 50%+ squares empty hote hain!

Analogy: 64 **dabbe** check karo. Khaali dabbe skip kar do, sirf **samaan wale** dekho. 📦

Step 4: PST Selection Logic

typescript

```
if (piece.type === 'k' && endgame) {  
  // King ke Liye SPECIAL table  
  pstVal = piece.color === 'w'  
    ? KING_ENDGAME_TABLE[r][c]  
    : KING_ENDGAME_TABLE[7 - r][c];  
} else {  
  // Normal PST
```

```
const table = PST[piece.type];
pstVal = piece.color === 'w'
  ? table[r][c]
  : table[7 - r][c];
}
```

🎯 MIRROR TRICK — Black Ke Liye `7 - r`

Ye sabse important concept hai!

Tables **White perspective** se likhi hain. Black ke liye **vertically flip** karni padti hai.

🎯 Visual Example:

White Knight at `g1` (Row 7, Col 6):

text

```
table[7][6] = -40
```

Wait, `g1` = Row 7 (Rank 1), Col 6 (g file). Knight PST mein:

text

```
Row 7: [-50, -40, -30, -30, -30, -30, -40, -50]
                ↑
            Col 6 = -40
```

Result: White knight `g1` pe = `-40` (bad — corner)

Black Knight at `g8` (Row 0, Col 6):

text

```
table[7 - 0][6] = table[7][6] = -40
```

Same value! Kyunki black knight bhi apne "g1" (mirrored) pe hai.

🎯 Mirroring Diagram:

text

White Perspective:	Black Perspective (Mirrored):
Rank 8 (Row 0)	Rank 1 (Row 7)
Rank 7 (Row 1)	Rank 2 (Row 6)
Rank 6 (Row 2)	Rank 3 (Row 5)
Rank 5 (Row 3)	Rank 4 (Row 4)
Rank 4 (Row 4)	Rank 5 (Row 3)
Rank 3 (Row 5)	Rank 6 (Row 2)
Rank 2 (Row 6)	Rank 7 (Row 1)
Rank 1 (Row 7)	Rank 8 (Row 0)

Mirror formula: `black_row = 7 - white_row`

Analogy: Tum **aaine (mirror)** ke saamne khade ho. Tumhara **left hand**, reflection ka **right hand** dikhta hai. Isi tarah, black ke liye **row ulta** karni padti hai. 🪞

🎯 Column Mirror Kyu Nahi?

Kyu sirf row, col nahi?

Kyunki chess **vertically symmetric** hai (queenside vs kingside different hain, but **left-right same**). Sirf **top-bottom** flip chahiye.

Wait, actually **col mirror bhi hoti hai** in some engines, but this version mein **sirf row** mirror karte hain. Kyunki **e1 vs d1** important ho sakta hai (king position), but generally symmetry acceptable.

Hmm actually — ye **correct** hai kyunki:

- **a1** aur **h1** same value hain (both corners)
- **d1** aur **e1** different ho sakte hain but usually similar

Note: Advanced engines mein **col mirror bhi** hoti hai (`7 - c`).

Step 5: Accumulate Scores

typescript

```
const totalVal = materialVal + pstVal;
if (piece.color === 'w') whiteScore += totalVal;
```

```
else blackScore += totalVal;
```

White pieces = whiteScore mein add

Black pieces = blackScore mein add

Kyu separate? Taaki akhir mein **difference** le sakein.

Step 6: Side-to-Move Perspective (Critical!)

typescript

```
const score = whiteScore - blackScore;  
return game.turn() === 'w' ? score : -score;
```

Ye Line Bhoolna = Engine Suicidal!

Negamax contract: Score hamesha current player ke liye **positive** hona chahiye.

Agar Black ki turn hai:

- White +300 hai (White aage hai)
- Black ke perspective se = **-300** (Black peeche hai)
- Isliye **flip** karo!

Analogy: Tum **cricket** khel rahe ho. Agar tumhare paas 150 runs hain, aur opponent ke paas 200, toh:

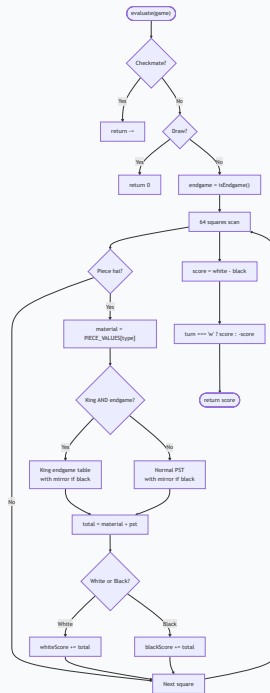
- Tumhare perspective: **-50** (peeche ho)
- Opponent ke perspective: **+50** (aage hai)

Same score, **different perspective!** 

Flowchart:

Diagram

Code



🔧 PART 6 — Helper Functions

6.1 evaluateMaterial() — Sirf Material

typescript

```
export function evaluateMaterial(game: Chess): number {
  if (game.isCheckmate()) return -Infinity;
  if (game.isDraw()) return 0;

  let material = 0;
  const board = game.board();
  for (let r = 0; r < 8; r++) {
    for (let c = 0; c < 8; c++) {
      const piece = board[r][c];
      if (piece) {
        const val = PIECE_VALUES[piece.type] ?? 0;
        material += piece.color === 'w' ? val : -val;
      }
    }
  }
  return game.turn() === 'w' ? material : -material;
}
```

Difference From `evaluate()` :

Feature	<code>evaluate()</code>	<code>evaluateMaterial()</code>
Material	✓	✓
PST bonus	✓	✗
Endgame detection	✓	✗

Kyu? Testing ke liye. Agar PST buggy hai, toh **pure material** check karke debug kar sakte hain.

Single variable `material` use karta hai (do alag `whiteScore` / `blackScore` ki jagah) — **simpler**.

6.2 `evaluateBoard()` — FEN Helper

typescript

```
export function evaluateBoard(fen: string): number {
  const game = new Chess(fen);
  return evaluate(game);
}
```

Ek-liner! FEN string se `Chess` object banao, aur `evaluate()` call karo.

Kab use? External scripts, tests, ya jab sirf **FEN** available ho (game object nahi).

6.3 `evaluateWhitePerspective()` — GUI Ke Liye

typescript

```
export function evaluateWhitePerspective(game: Chess): number {
  if (game.isCheckmate()) {
    return game.turn() === 'w' ? -Infinity : Infinity;
  }
  if (game.isDraw()) return 0;

  const sideToMoveScore = evaluate(game);
```

```
return game.turn() === 'w' ? sideToMoveScore : -sideToMoveScore;
}
```

Ye Kyu Alag?

EvalBar UI mein hamesha **White ke perspective** se score dikhata hai. But `evaluate()` **side-to-move** perspective deta hai.

Toh ye function `evaluate()` ko call karke wapas flip karta hai!

Checkmate Special Case:

```
typescript

if (game.isCheckmate()) {
  return game.turn() === 'w' ? -Infinity : Infinity;
}
```

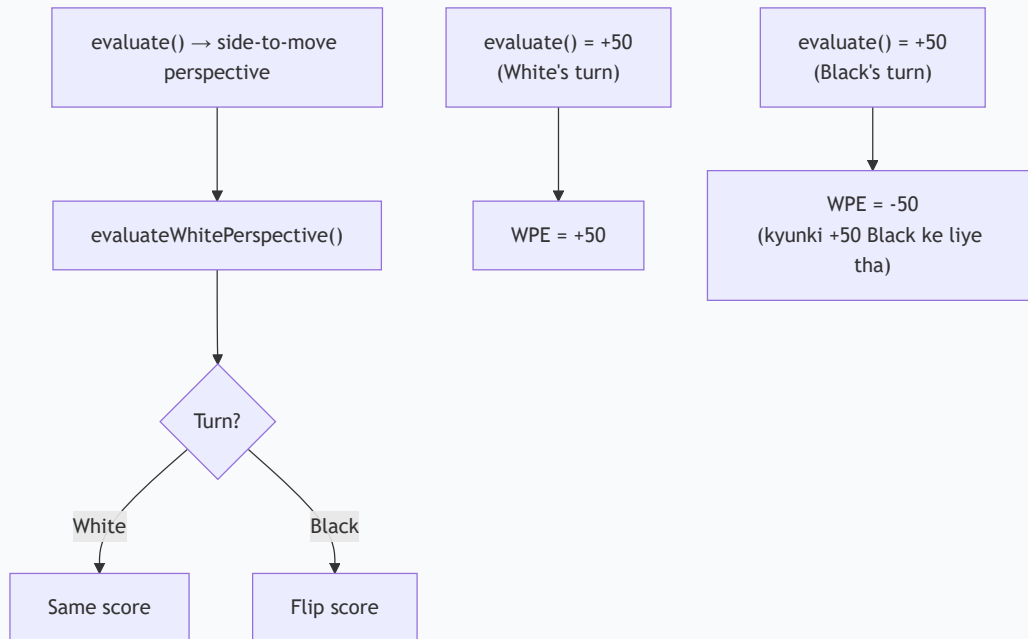
Same check as evaluate, but **White's perspective**:

- White ka turn hai aur checkmate = **White haara** = `-Infinity`
- Black ka turn hai aur checkmate = **Black haara** = `+Infinity`

Visual Flow:

Diagram

Code



🎯 Example:

Position: White has extra pawn, Black's turn.

text

`evaluate()` → -100 (Black ke liye +100, matlab White peeche... wait)

Hmm, let me redo:

White has extra pawn (White +100). Black's turn.

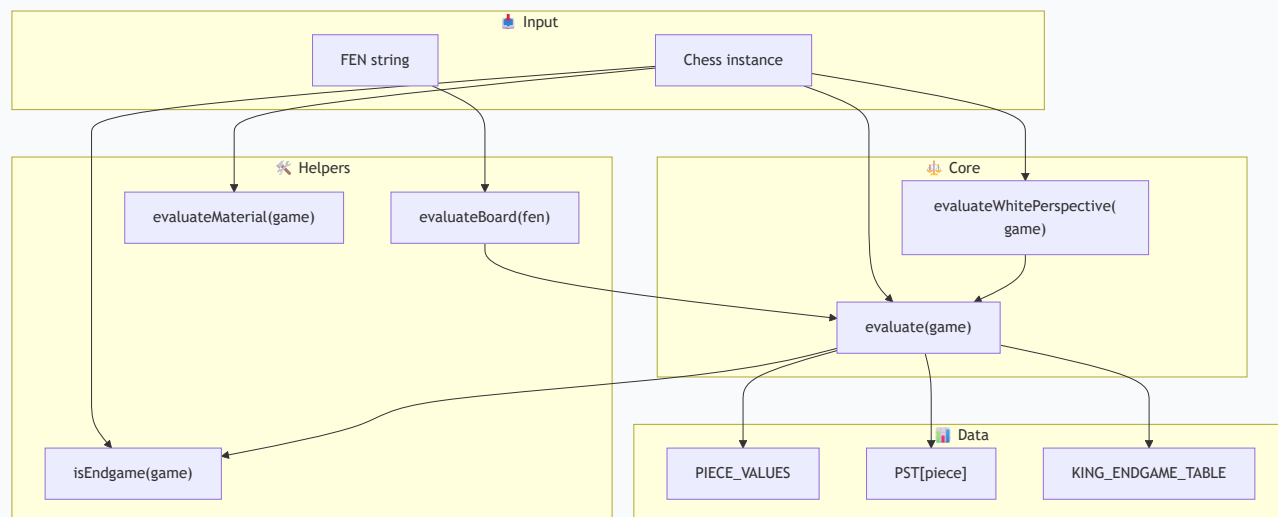
- `evaluate()` = side-to-move = Black ke perspective
- Black ke perspective: White +100 hai, toh Black **-100** hai
- `evaluate()` returns -100
- `evaluateWhitePerspective()` :
 - `sideToMoveScore` = -100
 - `turn == 'b'` → flip → `-(-100) = +100`
 - Return **+100** ✅ (White ke perspective)

Perfect!

🎯 Ab Saare Functions Ka Summary

Diagram

Code



🐛 Gotchas Is File Mein

⚠️ Gotcha #1: Perspective Flip Bhoolna

typescript

// ❌ GALAT

```
return whiteScore - blackScore;
```

// ✅ SAHI

```
const score = whiteScore - blackScore;
```

```
return game.turn() === 'w' ? score : -score;
```

Sabse common bug! Engine suicide moves chalega.

⚠️ Gotcha #2: Black Ke Liye Mirror Bhoolna

typescript

```
// ❌ GALAT
pstVal = table[r][c]; // Black ke liye bhi same

// ✅ SAHI
pstVal = piece.color === 'w'
  ? table[r][c]
  : table[7 - r][c]; // Black ke liye mirror
```

⚠️ Gotcha #3: King Endgame Table Ka Alag Handling

typescript

```
if (piece.type === 'k' && endgame) {
  // Special table
} else {
  // Normal PST
}
```

King ke liye **do tables** hain. Endgame mein different behavior.

⚠️ Gotcha #4: ?? 0 Fallback

typescript

```
const materialVal = PIECE_VALUES[piece.type] ?? 0;
```

Agar piece type **unknown** ho (impossible, but safety), toh `0`.

⚠️ Gotcha #5: -Infinity Comparisons

typescript

```
if (game.isCheckmate()) return -Infinity;
```

Agar search mein `-Infinity` handle na kare, toh **NaN** propagate ho sakta hai. `negamax` mein `maxScore = -Infinity` initial hai, toh careful rehna.

🎓 Ek Line Mein Summary

Ye file engine ki aankhein hain 🏴‍☠️ — `PIECE_VALUES` material count karti hai, `PST` tables batati hain ki pieces kahan acha perform karte hain (knight center mein +20, corner mein -50), `KING_ENDGAME_TABLE` king ko endgame mein active karti hai, `evaluate()` sab jodke side-to-move perspective ka score deta hai, aur `evaluateWhitePerspective()` GUI ke liye White's perspective return karta hai.

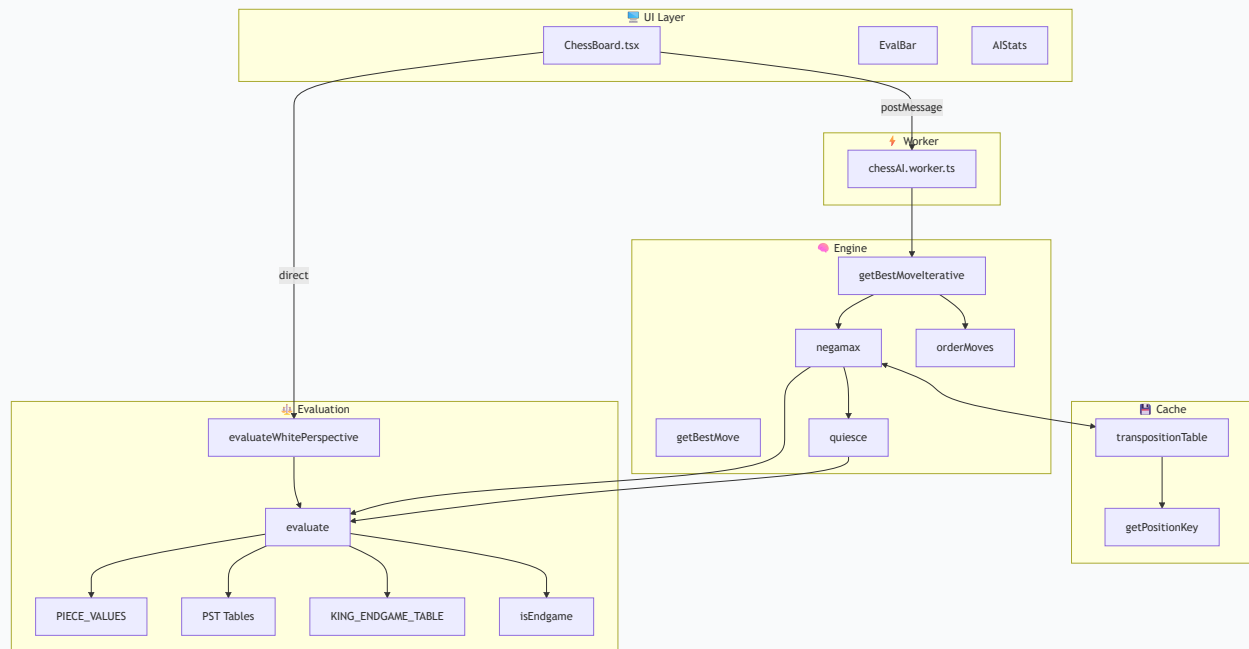
💡 Improvement Ideas

Idea	Kahan Change Karo
Pawn structure	Doubled, isolated, passed pawns evaluate karo
Bishop pair bonus	Dono bishops = +30
King safety	Pawn shield, open files near king
Mobility	Kitne legal moves hain
Rook on open file	Bonus add karo
Tempo	Small bonus for side to move
Incremental eval	Poora board scan ke bajaye delta
Tapered eval	Opening vs endgame ke beech smooth blend

📊 Poore Project Ka Final Picture

Diagram

Code



🎉 Mubarak Ho! Poora Project Cover Ho Gaya!

Ab tak humne dekha:

1. ✅ ChessBoard.tsx — UI + handlers
2. ✅ chessAI.worker.ts — Worker bridge
3. ✅ chessAI.ts — Engine core
4. ✅ transposition.ts — Cache
5. ✅ evaluation.ts — Scoring (ye file)

Ab tumhare paas poora mental model hai:

User move karta hai → ChessBoard validate karta hai → Worker ko FEN bhejta hai → Engine iterative deepening chalata hai → Negamax har position pe evaluate() call karta hai → evaluate() material + PST se score deta hai → TT cache se reuse hota hai → Best move wapas UI ko jaata hai bina freeze kiye! 🚀

Bhai, agar koi **specific concept** aur detail mein chahiye, bata dena:

- PST values **kahan se aayi?** (chess theory + tuning)
- **Tapered evaluation** kya hai? (opening vs endgame blend)
- **Zobrist hashing** kya hai? (TT ke liye faster)
- **Aspiration windows** kya hai? (advanced search)

Main **wahi ek topic** le ke **poora deep dive** karunga! 💪

This response is AI-generated, for reference only.